클라이언트 서버 프로그래밍 과제

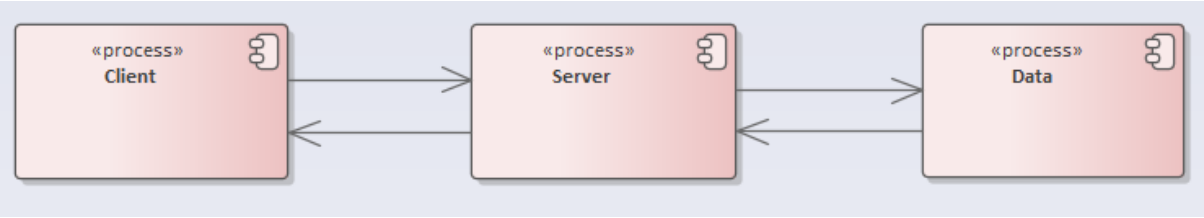
60181924 이우성

0. 개요

이 보고서는 클라이언트 서버 프로그래밍을 이용한 학사 관리 시스템에 대해서 설명한다. 첫번째로는 학사 관리 시스템의 기능과 각 프로세스를 어떻게 설계했는지 설명한다. 설계는 EA와 소스 코드를 통해서 기술한다. 두번째로는 소스 코드를 실행한 결과를 기술한다. 코드를 실행했을 때 나타날 수 있는 경우의 수를 상세하게 모두 기술한다. 마지막 세번째로는 결론을 통해 보완점과 느낀 점을 기술한다.

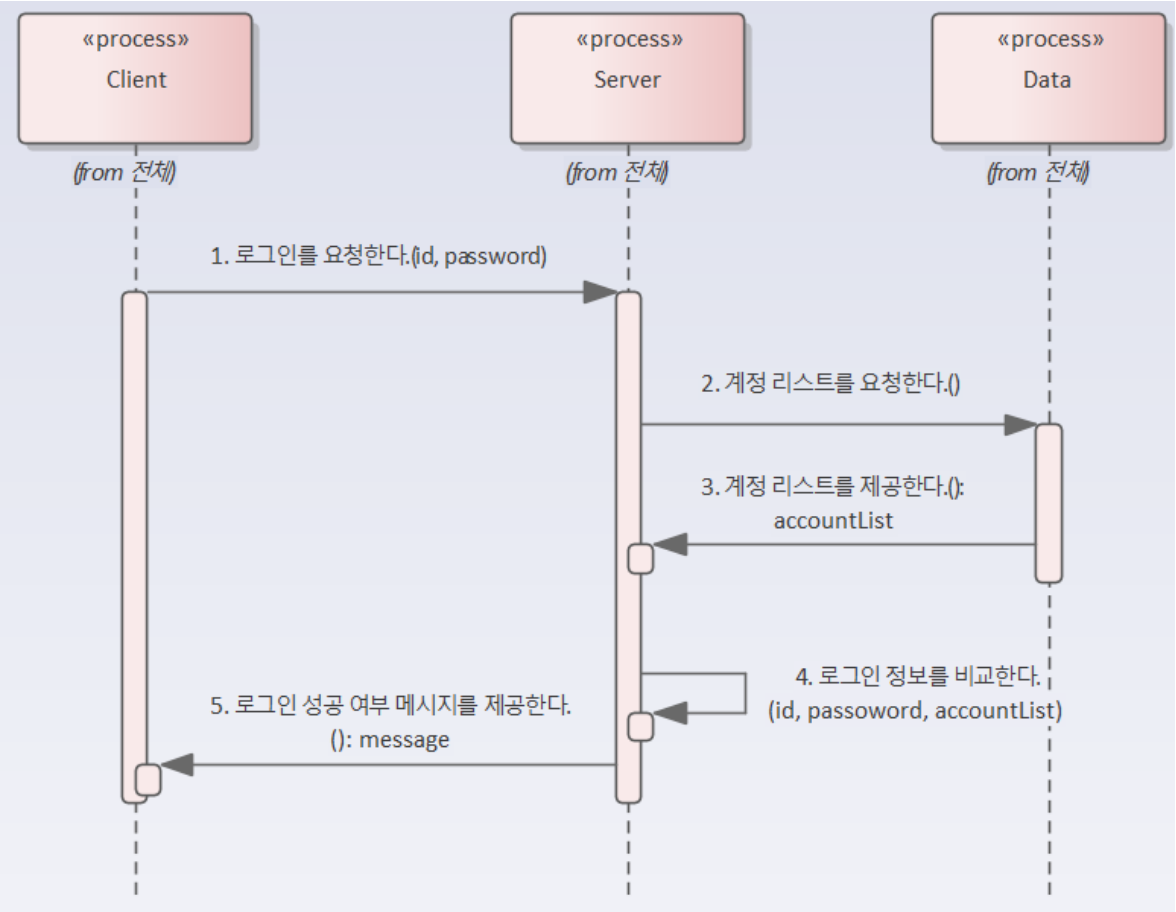
1. EA & 코드

학사 관리 시스템은 자바로 작성했으며, 프로세스 간의 통신을 위해서 GRPC 프레임워크를 사용하였다. 학사 관리 시스템은 로그인, 회원 가입, 학생 리스트 조회, 강의 리스트 조회, 학생 추가/삭제, 강의 추가/삭제, 수강신청 그리고 로깅 기능을 갖고 있다.

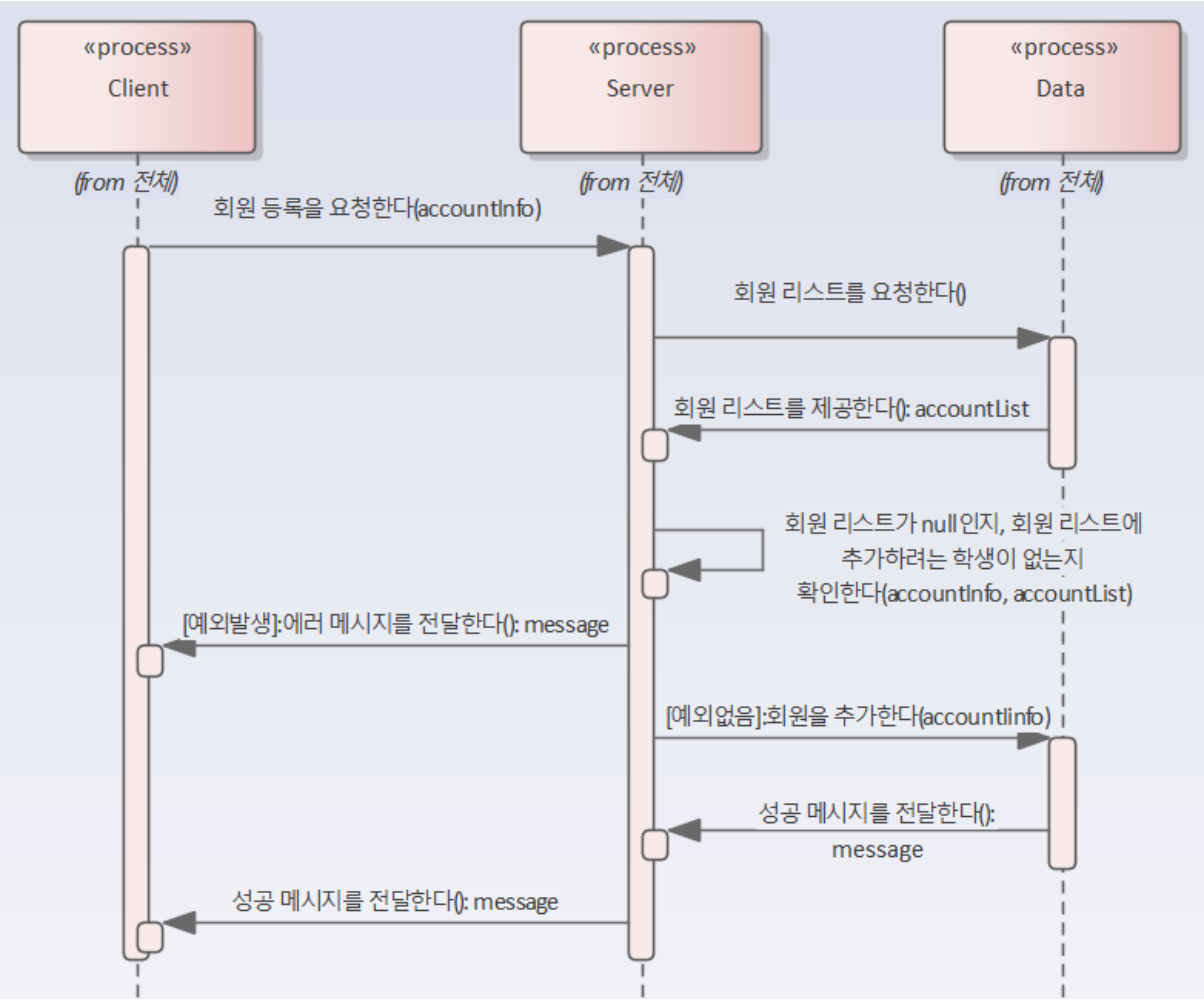


학사 관리 시스템은 3개의 프로세스가 통신하여 동작한다. Client는 사용자로부터 데이터를 입력받아 Server에게 데이터를 전달하면서 서비스를 요청하고, Server로부터 받은 데이터를 보기 좋게 가공하여 사용자에게 보여준다. Server는 Client로부터 받은 데이터와 요청에 따라 Data에게 데이터를 전달하고 서비스를 요청한다. 그리고 Data로부터 반환되는 데이터를 Client에게 전달한다. Server는 Client와 Data로부터 전달받은 데이터를 검사한다. 모든 예외검사/처리가 Server에서 이루어진다. 검사 결과 데이터에 문제가 있을 경우에는 예외를 발생시키고 에러 메시지를 Client에게 전달한다. Data는 Server로부터 받은 요구에 따라서 데이터를 처리하고 결과를 Server에게 전달한다.

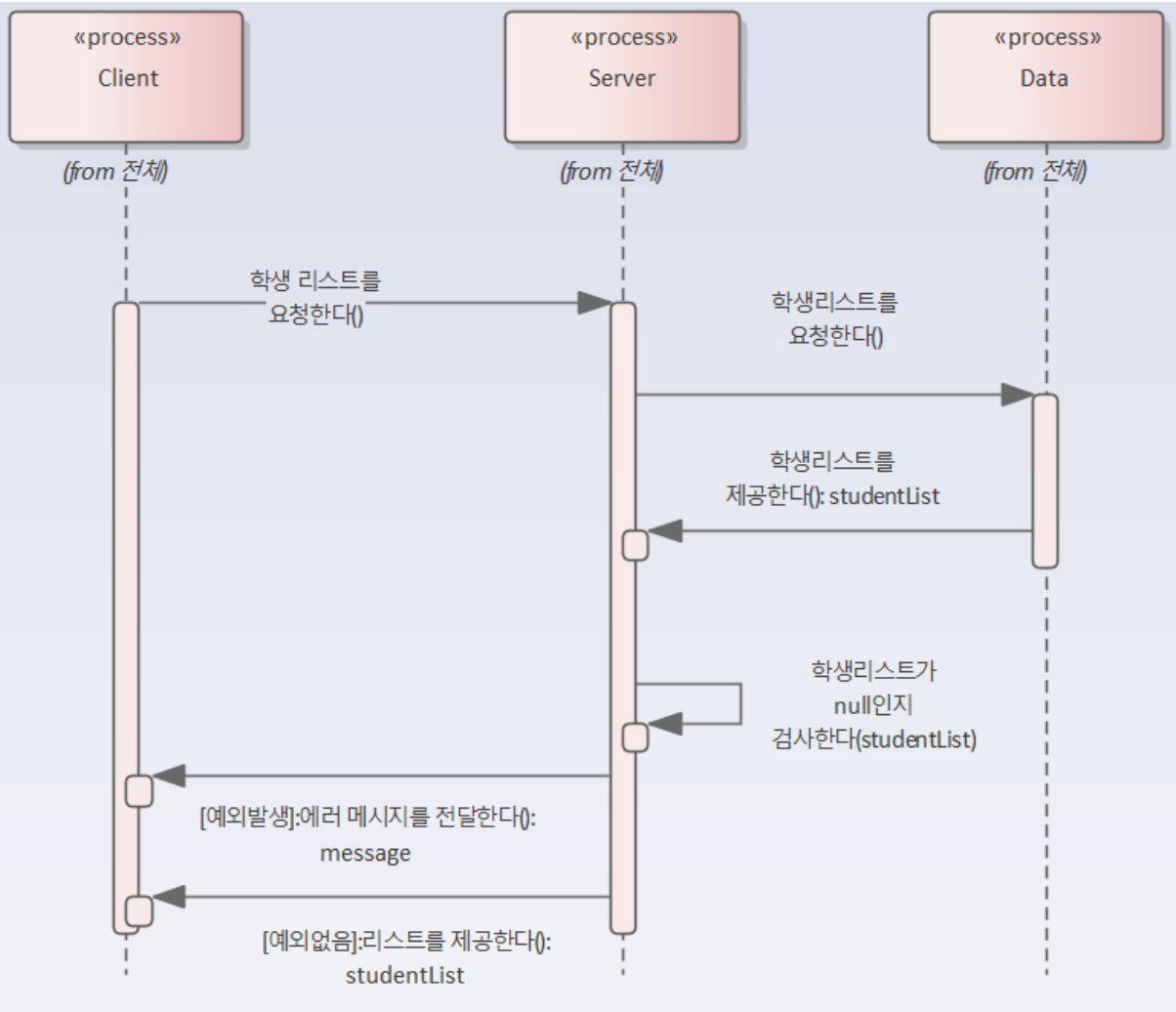
로그인 기능 시퀀스


«process»
Client
«process»
Server
«process»
Data
(from 전체)
(from 전체)
(from 전체)
1. 로그인를 요청한다.(id, password)
2. 계정 리스트를 요청한다.()
3. 계정 리스트를 제공한다.():
accountList
4. 로그인 정보를 비교한다.
(id, passoword, accountList)
5. 로그인 성공 여부 메시지를 제공한다.
(): message

회원가입 기능 시퀀스

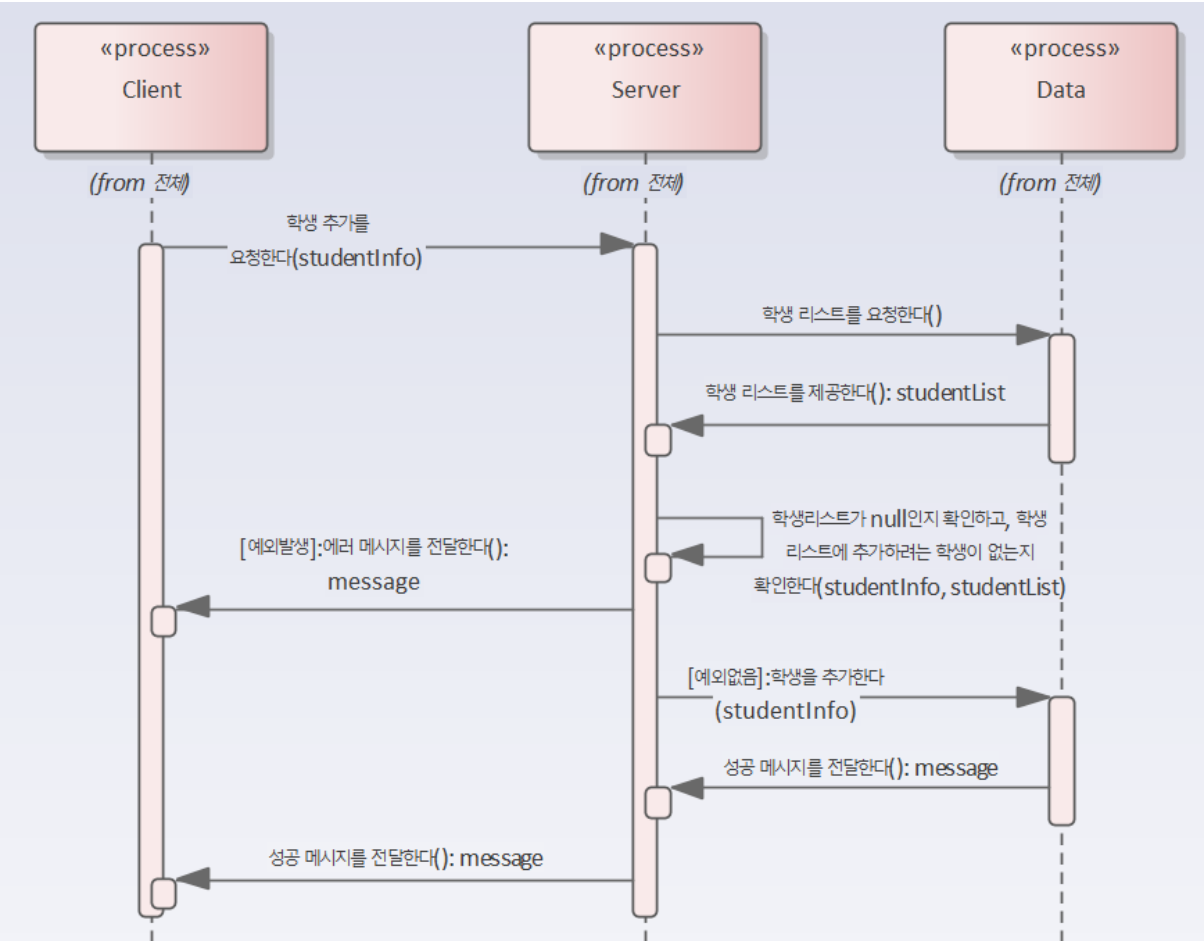

«process»
Client
«process»
Server
«process»
Data
(from 전체)
(from 전체)
(from 전체)
회원 등록을 요청한다(accountInfo)
회원 리스트를 요청한다()
회원 리스트를 제공한다(): accountList
회원 리스트가 null인지, 회원 리스트에
추가하려는 학생이 없는지
확인한다(accountInfo, accountList)
[예외발생]:에러 메시지를 전달한다(): message
[예외없음]:회원을 추가한다(accountInfo)
성공 메시지를 전달한다():
message
성공 메시지를 전달한다(): message

학생 리스트 조회 기능 시퀀스



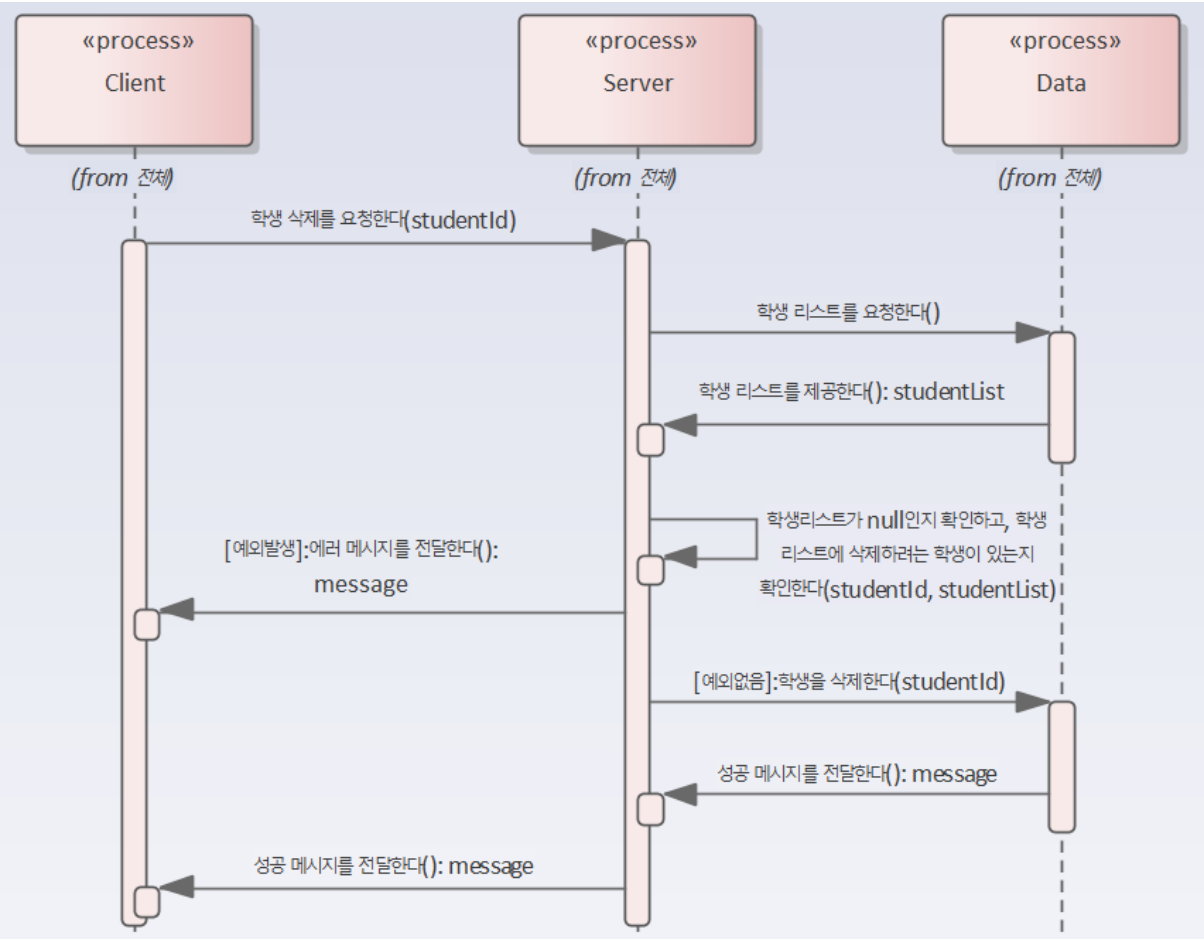
강의 리스트 조회 기능 시퀀스 역시 이것과 동일하다.

학생 추가 기능 시퀀스



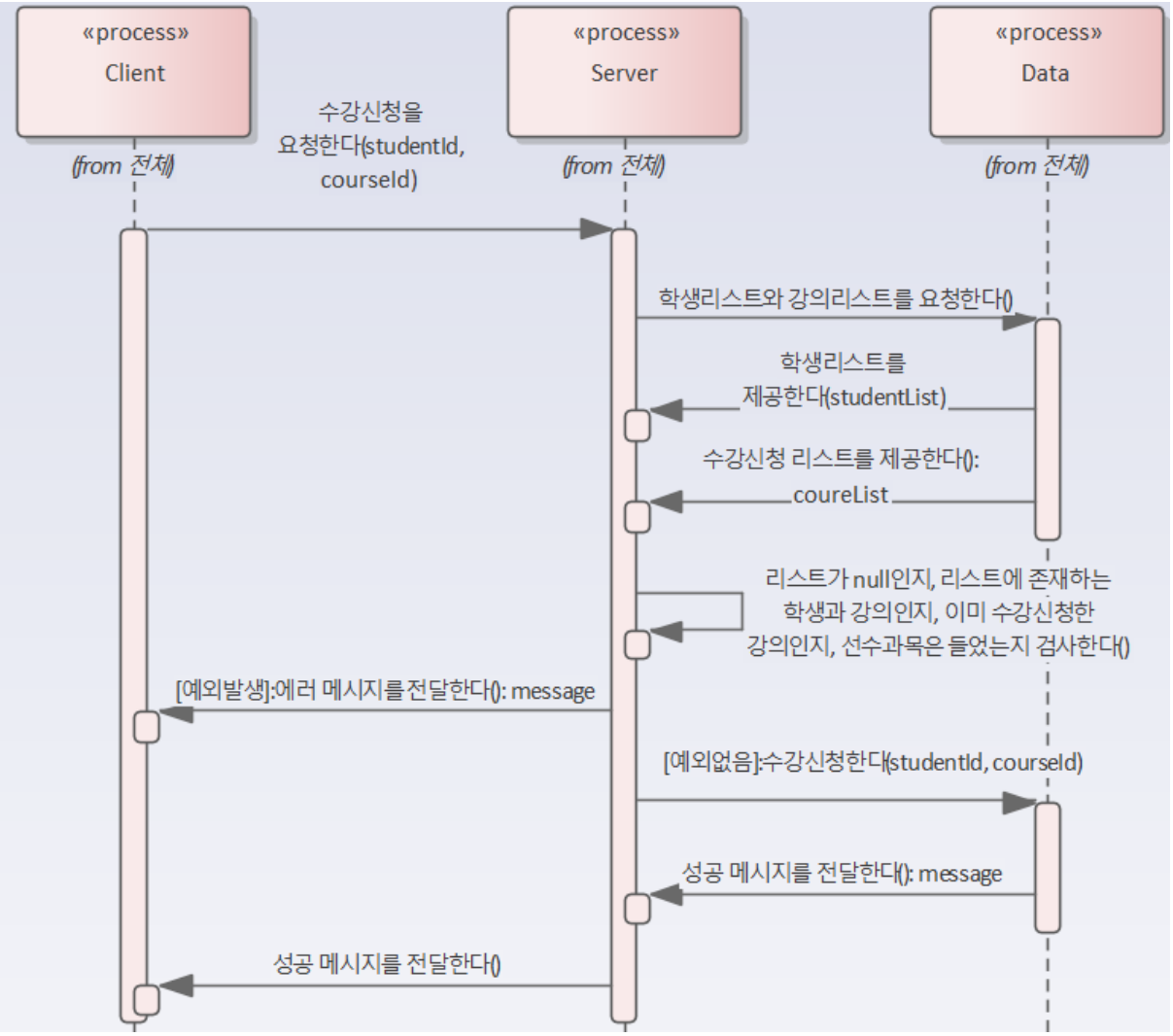
강의 추가 기능 시퀀스 역시 이것과 동일하다.

학생 삭제 기능 시퀀스

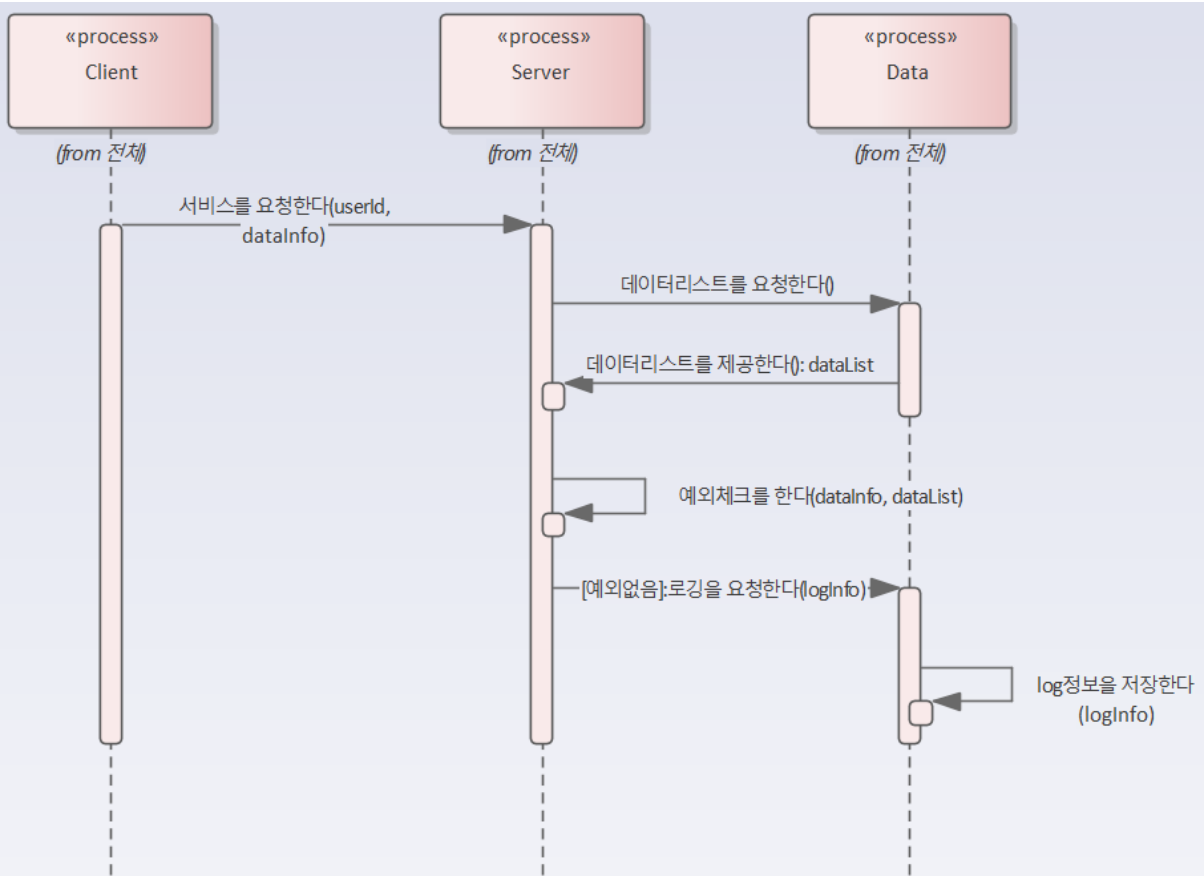


강의 삭제 기능 시퀀스 역시 이것과 동일하다.

수강 신청 기능 시퀀스


«process»
Client
«process»
Server
«process»
Data
(from 전체)
(from 전체)
(from 전체)
수강신청을
요청한다(studentId,
courseId)
학생리스트와 강의리스트를 요청한다()
학생리스트를
제공한다(studentList)
수강신청 리스트를 제공한다():
coureList
리스트가 null인지, 리스트에 존재하는
학생과 강의인지, 이미 수강신청한
강의인지, 선수과목은 들었는지 검사한다()
[예외발생]:에러 메시지를 전달한다(): message
[예외없음]:수강신청한다(studentId, courseId)
성공 메시지를 전달한다(): message
성공 메시지를 전달한다()

로깅 기능 시퀀스


«process»
Client
(from 전체)
«process»
Server
(from 전체)
«process»
Data
(from 전체)
서비스를 요청한다(userId, dataInfo)
데이터리스트를 요청한다()
데이터리스트를 제공한다(): dataList
예외체크를 한다(dataInfo, dataList)
[예외없음]:로깅을 요청한다(logInfo)
log정보을 저장한다 (logInfo)

Proto 파일

GRPC 코드를 생성하기 위해서 Proto파일을 작성하고 빌드하여 자동으로 코드를 생성하였다. Get, Add, Delete, Login로 서비스를 4개로 나누었다. 그리고 각 서비스에 필요한 함수를 정의하였다.

```
service GetService{
    rpc GetAllStudent(UserId) returns (StudentList);
    rpc GetAllCourse(UserId) returns (CourseList);
    rpc GetAllAccount(UserId) returns (AccountList);
}
```

```
message UserId{
    string userId = 1;
}
```

```
message StudentList{
    repeated Student student = 1;
    string exception = 2;
}

message CourseList{
    repeated Course course = 1;
    string exception = 2;
}

message AccountList{
    repeated Account account = 1;
    string exception = 2;
}
```

```
message Student{
    string studentId = 1;
    string name = 2;
    string department = 3;
    repeated string completedCourse = 4;
    repeated string reservedCourse = 5;
}

message Course{
    string courseId = 1;
    string professorName = 2;
    string courseName = 3;
    repeated string prerequisite = 4;
    repeated string reservedStudent = 5;
}

message Account{
    string id = 1;
    string password = 2;
}
```

GetService에서는 요청할 때 사용자의 아이디를 전달한다. 유저의 로깅 정보를 저장하기 위해서다. 그리고 데이터 리스트(StudetList, CourseList, AccountList)를 반환한다. 데이터 리스트에는 데이터들 뿐만 아니라 문자열 타입의 exception를 가지고 있다. 예외가 발생했을 시에 예외의 정보를 문자열로 exception에 담아서 반환한다. 예외가 발생하지 않으면 exception은 빈 채로 반환한다.

```
service AddService{
    rpc AddStudent(DataInfo) returns (ExceptionMessage);
    rpc AddCourse(DataInfo) returns (ExceptionMessage);
    rpc MakeReservation(DataInfo) returns(ExceptionMessage);
    rpc AddAccount(DataInfo) returns (ExceptionMessage);
    rpc AddLog(DataInfo) returns (Null);
}
```

```
message DataInfo{
    string userId = 1;
    string Info = 2;
}
```

```
message ExceptionMessage{
    string message = 1;
}
```

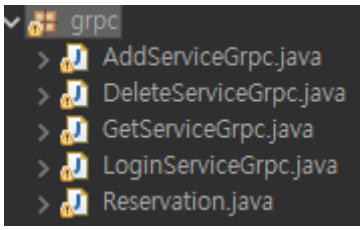
```
message Null{
}
```

AddService에서는 요청할 때 추가할 데이터의 정보를 DataInfo의 Info에 담고 사용자 아이디를 userId에 담아서 전달한다. 사용자 아이디는 로깅을 하기 위해서다. 예외가 발생할 경우 ExceptionMessage의 message에 예외 정보를 담아서 반환한다. 예외가 발생하지 않으면 message를 빈 채로 반환한다. AddLog는 단순히 log.txt에 로깅 정보를 쓰는 역할만 하기 때문에 Null을 반환한다.

```
service DeleteService{
    rpc DeleteStudent(DataInfo) returns (ExceptionMessage);
    rpc DeleteCourse(DataInfo) returns (ExceptionMessage);
}

service LoginService{
    rpc Login(DataInfo) returns (ExceptionMessage);
    rpc SignUp(DataInfo) returns (ExceptionMessage);
}
```

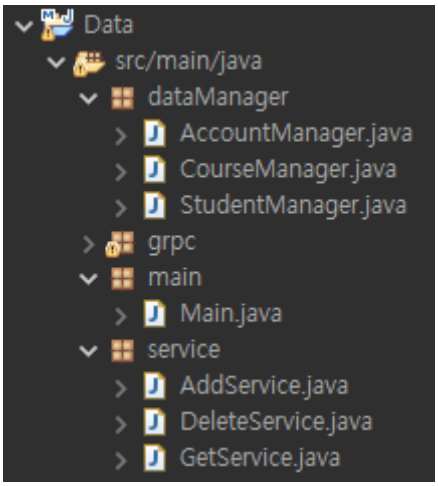
DeleteService는 요청할 때 삭제할 데이터의 정보를 DataInfo의 info에 담고 로깅을 위한 사용자의 아이디를 userId에 담아 요청한다. 예외 발생 시 ExceptionMessage의 message에 예외 내용을 넣어 반환하고 예외 미발생시 message를 빈 채로 넣어 반환한다.

LoginService는 요청할 때 로그인/회원가입 할 데이터의 정보를 DataInfo의 info에 담고 로깅을 위한 사용자의 아이디를 userId에 담아 요청한다. 예외 발생 시 ExceptionMessage의 message에 예외 내용을 넣어 반환하고 예외 미발생시 message를 빈 채로 넣어 반환한다.



프로토 파일을 통해서 자동 생성된 클래스다. 이 클래스들은 gprc패키지에 담아서 Client, Server, Data가 공통적으로 상용한다. 각각의 프로세스에서 이 클래스들을 상속받은 서비스 클래스를 만들고 함수를 재정의하여 사용한다.

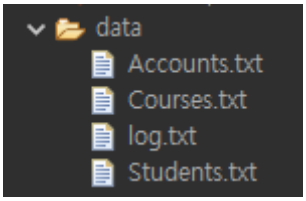
Data의 클래스



dataManger패키지의 클래스는 Student, course, account의 데이터리스트를 생성하고 데이터를 조회 및 수정하는 역할을 한다.

service패키지의 클래스는 grpc패키지의 클래스를 상속받아 함수를 재정의하여 사용한다.

Main 클래스는 서버를 만들고 데이터매니저를 붙인 서비스를 서버에 띄운다.



데이터리스트를 최초에 생성할 때 쓸 txt파일이다. log파일은 로깅 정보를 저장하는 txt파일이다.

AccountManager

```java
public class AccountManager {

    private AccountList.Builder accountList;

    public AccountManager(String fileName) throws IOException {
        this.accountList = AccountList.newBuilder();
        BufferedReader objCourseFile = new BufferedReader(new FileReader(fileName));
        while (objCourseFile.ready()) {
            String accountInfo = objCourseFile.readLine();
            if (!accountInfo.equals("")) this.accountList.addAccount(this.setAccount(accountInfo));
        }
        this.accountList.setException("");
        objCourseFile.close();
    }

    public Account.Builder setAccount(String inputString) {
        Account.Builder account = Account.newBuilder();
        StringTokenizer stringTokenizer = new StringTokenizer(inputString);
        account.setId(stringTokenizer.nextToken());
        account.setPassword(stringTokenizer.nextToken());
        return account;
    }
```

```java
    public AccountList.Builder getAccountList(){
        return this.accountList;
    }

    public void addAccount(String accountInfo){
        this.accountList.addAccount(this.setAccount(accountInfo));
    }
}
```

생성자에서 파일 경로를 전달받아서 accountList를 생성한다. 조회, 추가 기능이 있다.

CourseManager

```java
public class CourseManager {
    protected CourseList.Builder courseList;

    public CourseManager(String fileName) throws IOException {
        this.courseList = CourseList.newBuilder();
        BufferedReader objCourseFile = new BufferedReader(new FileReader(fileName));
        while (objCourseFile.ready()) {
            String courseInfo = objCourseFile.readLine();
            if (!courseInfo.equals("")) this.courseList.addCourse(this.setCourse(courseInfo));
        }
        this.courseList.setException("");
        objCourseFile.close();
    }

    public Course.Builder setCourse(String inputString) {
        Course.Builder course = Course.newBuilder();
        StringTokenizer stringTokenizer = new StringTokenizer(inputString);
        course.setCourseId(stringTokenizer.nextToken());
        course.setProfessorName(stringTokenizer.nextToken());
        course.setCourseName(stringTokenizer.nextToken());
        while (stringTokenizer.hasMoreTokens()) {
            course.addPrerequisite(stringTokenizer.nextToken());
        }
        return course;
    }
```

```java
    public CourseList.Builder getCourseList(){
        return this.courseList;
    }
    public void addCourse(String courseInfo){
        this.courseList.addCourse(this.setCourse(courseInfo));
    }
    public void deleteCourse(String courseId){
        for (int i = 0; i < this.courseList.getCourseCount(); i++) {
            Course.Builder course = this.courseList.getCourseBuilder(i);
            if (course.getCourseId().equals(courseId)) this.courseList.removeCourse(i); return;
        }
    }
    public void makeReservation(String reservationInfo) {
        StringTokenizer stringTokenizer = new StringTokenizer(reservationInfo);
        String studentId = stringTokenizer.nextToken();
        String courseId = stringTokenizer.nextToken();
        for (int i = 0; i < this.courseList.getCourseCount(); i++) {
            Course.Builder course = this.courseList.getCourseBuilder(i);
            if (course.getCourseId().equals(courseId)) course.addReservedStudent(studentId); return;
        }
    }
}
```

생성자에서 파일 경로를 전달받아 courseList를 생성한다. 조회, 추가, 삭제, 신청 기능이 있다.

StudentManager

```java
public class StudentManager {

    private StudentList.Builder studentList;

    public StudentManager(String fileName) throws IOException {
        this.studentList = StudentList.newBuilder();
        BufferedReader objStudentFile = new BufferedReader(new FileReader(fileName));
        while (objStudentFile.ready()) {
            String stuInfo = objStudentFile.readLine();
            if (!stuInfo.equals("")) this.studentList.addStudent(this.setStudent(stuInfo));
        }
        this.studentList.setException("");
        objStudentFile.close();
    }

    public Student.Builder setStudent(String inputString) {
        Student.Builder student = Student.newBuilder();
        StringTokenizer stringTokenizer = new StringTokenizer(inputString);
        student.setStudentId(stringTokenizer.nextToken())
        .setName(stringTokenizer.nextToken() + " " + stringTokenizer.nextToken())
        .setDepartment(stringTokenizer.nextToken());
        while (stringTokenizer.hasMoreTokens()) student.addCompletedCourse(stringTokenizer.nextToken());
        return student;
    }
```

```java
    public StudentList.Builder getStudentList() {
        return this.studentList;
    }

    public void addStudent(String studentInfo){
        this.studentList.addStudent(this.setStudent(studentInfo));
    }

    public void deleteStudent(String studentId){
        for (int i = 0; i < this.studentList.getStudentCount(); i++) {
            Student.Builder student = this.studentList.getStudentBuilder(i);
            if (student.getStudentId().equals(studentId)) this.studentList.removeStudent(i); return;
        }
    }

    public void makeReservation(String reservationInfo) {
        StringTokenizer stringTokenizer = new StringTokenizer(reservationInfo);
        String studentId = stringTokenizer.nextToken();
        String courseId = stringTokenizer.nextToken();
        for (int i = 0; i < this.studentList.getStudentCount(); i++) {
            Student.Builder student = this.studentList.getStudentBuilder(i);
            if (student.getStudentId().equals(studentId)) student.addReservedCourse(courseId); return;
        }
    }
}
```

생성자에서 파일 경로를 전달받아 studentList를 생성한다. 조회, 추가, 삭제, 신청 기능이 있다.

AddService

```java
public class AddService extends AddServiceImplBase{

    private StudentManager studentManager;
    private CourseManager courseManager;
    private AccountManager accountManager;

    public AddService(StudentManager studentManager, CourseManager courseManager, AccountManager accountManager) {
        this.studentManager = studentManager;
        this.courseManager = courseManager;
        this.accountManager = accountManager;
    }

    @Override
    public void addStudent(DataInfo request, StreamObserver<ExceptionMessage> responseObserver) {
        this.studentManager.addStudent(request.getInfo());
        responseObserver.onNext(ExceptionMessage.newBuilder().build());
        responseObserver.onCompleted();
    }

    @Override
    public void addCourse(DataInfo request, StreamObserver<ExceptionMessage> responseObserver) {
        this.courseManager.addCourse(request.getInfo());
        responseObserver.onNext(ExceptionMessage.newBuilder().build());
        responseObserver.onCompleted();
    }

    @Override
    public void makeReservation(DataInfo request, StreamObserver<ExceptionMessage> responseObserver) {
        this.studentManager.makeReservation(request.getInfo());
        this.courseManager.makeReservation(request.getInfo());
        responseObserver.onNext(ExceptionMessage.newBuilder().build());
        responseObserver.onCompleted();
    }
```

```java
    @Override
    public void addAccount(DataInfo request, StreamObserver<ExceptionMessage> responseObserver) {
        this.accountManager.addAccount(request.getInfo());
        responseObserver.onNext(ExceptionMessage.newBuilder().build());
        responseObserver.onCompleted();
    }

    @Override
    public void addLog(DataInfo request, StreamObserver<Null> responseObserver) {
        try {
            BufferedWriter logWriter = new BufferedWriter(new FileWriter(new File("data/log.txt"), true));
            logWriter.newLine();
            logWriter.write(request.getUserId()+" "+request.getInfo());
            logWriter.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
        responseObserver.onNext(Null.newBuilder().build());
        responseObserver.onCompleted();
    }
}
```

AddServiceImplBase를 상속받아 함수를 재정의하여 사용한다. 생성자를 통해 StudentManager, CourseManager, AccountManager를 받아서 함수에서 데이터를 추가하는데 사용한다. 각 함수는 Server로부터 전달받은 DataInfo의 데이터를 추가하고, ExceptionMessage를 Server로 반환한다. 단 예외검사/처리는 Server에서 하기 때문에 Data에서는 비어있는 ExceptionMessage를 반환한다. addLog함수는 전달받은 DataInfo에 있는 로깅정보를 data/log.txt에 적는다.

DeleteService

```java
public class DeleteService extends DeleteServiceImplBase{

    private StudentManager studentManager;
    private CourseManager courseManager;

    public DeleteService(StudentManager studentManager, CourseManager courseManager) {
        this.studentManager = studentManager;
        this.courseManager = courseManager;
    }
    @Override
    public void deleteStudent(DataInfo request, StreamObserver<ExceptionMessage> responseObserver) {
        this.studentManager.deleteStudent(request.getInfo());
        responseObserver.onNext(ExceptionMessage.newBuilder().build());
        responseObserver.onCompleted();
    }

    @Override
    public void deleteCourse(DataInfo request, StreamObserver<ExceptionMessage> responseObserver) {
        this.courseManager.deleteCourse(request.getInfo());
        responseObserver.onNext(ExceptionMessage.newBuilder().build());
        responseObserver.onCompleted();
    }

}
```

DeleteServiceImplBase를 상속받아 함수를 재정의하여 사용한다. 생성자를 통해 StudentManager, CourseManager를 받아서 함수에서 데이터를 삭제하는데 사용한다. 각 함수는 Server로부터 전달받은 DataInfo의 데이터를 데이터리스트에서 삭제하고, ExceptionMessage를 Server로 반환한다. 단 예외검사/처리는 Server에서 하기 때문에 Data에서는 비어있는 ExceptionMessage를 반환한다.

GetService

```java
public class GetService extends GetServiceImplBase{

    private StudentManager studentManager;
    private CourseManager courseManager;
    private AccountManager accountManager;

    public GetService(StudentManager studentManager, CourseManager courseManager, AccountManager accountManager) {
        this.studentManager = studentManager;
        this.courseManager = courseManager;
        this.accountManager = accountManager;
    }

    @Override
    public void getAllStudent(UserId request, StreamObserver<StudentList> responseObserver) {
        StudentList.Builder studentList = this.studentManager.getStudentList();
        responseObserver.onNext(studentList.build());
        responseObserver.onCompleted();
    }

    @Override
    public void getAllCourse(UserId request, StreamObserver<CourseList> responseObserver) {
        CourseList.Builder courseList = this.courseManager.getCourseList();
        responseObserver.onNext(courseList.build());
        responseObserver.onCompleted();
    }

    @Override
    public void getAllAccount(UserId request, StreamObserver<AccountList> responseObserver) {
        AccountList.Builder accountList = this.accountManager.getAccountList();
        responseObserver.onNext(accountList.build());
        responseObserver.onCompleted();
    }
}
```

GetServiceImplBase를 상속받아 함수를 재정의하여 사용한다. 생성자를 통해 StudentManager,CourseManager, AccountManager를 받아서 함수에서 데이터를 조회하는데 사용한다. 각 함수는 데이터리스트를 반환한다. 데이터리스트 안에는 exceptionMessage가 있다. 예외검사/처리는 Server에서 하기 때문에 데이터리스트의 exceptionMessage는 비운 채로 반환한다.

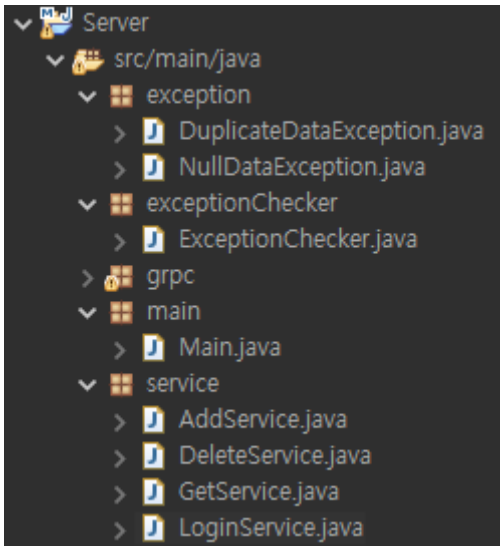
Main

```java
public class Main {

    public static void main(String[] args) throws FileNotFoundException, IOException, InterruptedException{
        StudentManager studentManager = new StudentManager("data/Students.txt");
        CourseManager courseManager = new CourseManager("data/Courses.txt");
        AccountManager accountManager = new AccountManager("data/Accounts.txt");

        Server server = ServerBuilder.forPort(8080)
                .addService(new AddService(studentManager, courseManager, accountManager))
                .addService(new DeleteService(studentManager, courseManager))
                .addService(new GetService(studentManager, courseManager, accountManager))
                .build();
        server.start();
        System.out.println("data is ready!!");
        System.out.println("port : 8080");
        server.awaitTermination();
    }
}
```

StudentManger, CourseManger, AccountManager에 생성자 파라미터로 파일 경로를 주고 실체화한다. 그리고 AddService, DeleteService, GetService에 StudentManger, CourseManger, AccountManager를 생성자 파라미터로 넣어 실체화한다. 실체화한 AddService, DeleteService,

GetService를 붙인 서버를 만들고, 서버를 시작한다.

Server의 클래스



Exception 패키지에는 예외 클래스가 있다. DuplicateDataException은 중복이 발생했을 때 던지는 외 클래스다. NullDataException 있어야 하는 데이터가 없을 때 던지는 예외 클래스다.

ExceptionChecker는 데이터리스트와 데이터를 전달받아 예외를 체크하는 클래스다. 데이터리스트가 존재하는지, 데이터리스트에 데이터가 존재하는지 혹은 중복되는지를 체크한다.

Main은 Data의 서버에 접속할 stub을 만들고 Client가 접속할 서버를 띄운다.

Service 패키지의 클래스는 grpc패키지의 클래스를 상속받아 함수를 재정의하여 사용한다.

DuplicateDataException

```java
public class DuplicateDataException extends Exception{
    private static final long serialVersionUID = 1L;
    private String errorMessage;

    public DuplicateDataException(String errorMessage) {
        super(errorMessage);
        this.errorMessage = errorMessage;
    }

    public String getErrorMessage() {
        return this.errorMessage;

    }
}
```

있어야 할 데이터가 없을 때 던지는 클래스다. Exception을 상속받는다. 생성자에서 errorMessage를 받아 에러메시지와 함께 에러 상세정보를 띄운다. 에러메시지를 문자열로 반환하는 getErrorMessage 함수를 가지고 있다.

NullDataException

```java
public class NullDataException extends Exception{
    private static final long serialVersionUID = 1L;
    private String errorMessage;
    public NullDataException(String errorMessage) {
        super(errorMessage);
        this.errorMessage = errorMessage;
    }

    public String getErrorMessage() {
        return this.errorMessage;

    }
}
```

데이터가 중복될 때 던지는 클래스다 Exception을 상속받는다. 생성자에서 errorMessage를 받아 에러메시지와 함께 에러 상세정보를 띄운다. 에러메시지를 문자열로 반환하는 getErrorMessage 함수를 가지고 있다.

ExceptionChecker

```java
public class ExceptionChecker {

    public void checkNullData(StudentList studentList) throws NullDataException {
        if(studentList.getStudentCount()==0) throw new NullDataException("StudentList is null");
    }

    public void checkNullData(CourseList courseList) throws NullDataException {
        if(courseList.getCourseCount()==0) throw new NullDataException("CourseList is null");
    }

    public void checkNullData(AccountList accountList) throws NullDataException {
        if(accountList.getAccountCount()==0) throw new NullDataException("AccountList is null");
    }

    public void checkDuplicateId(StudentList studentList, String studentId) throws DuplicateDataException, NullDataException {
        this.checkNullData(studentList);
        for(int i = 0; i<studentList.getStudentCount(); i++) {
            if(studentList.getStudent(i).getStudentId().equals(studentId)) throw new DuplicateDataException("Student ID is duplicated");
        }
    }

    public void checkDuplicateId(CourseList courseList, String id) throws DuplicateDataException, NullDataException {
        this.checkNullData(courseList);
        for(int i = 0; i<courseList.getCourseCount(); i++) {
            if(courseList.getCourse(i).getCourseId().equals(id)) throw new DuplicateDataException("Course ID is duplicated");
        }
    }

    public void checkDuplicateId(AccountList accountList, String accountId) throws DuplicateDataException, NullDataException {
        this.checkNullData(accountList);
        for(int i = 0; i<accountList.getAccountCount(); i++) {
            if(accountList.getAccount(i).getId().equals(accountId)) throw new DuplicateDataException("Account Id is duplicated");
        }
    }
```

checkNullData 함수는 데이터리스트를 받아서 리스트가 비어 있으면 NullDataException에 에러 메시지를 담아 던진다.

checakDuplicatedId는 데이터리스트와 아이디를 받아 checkNullData 함수를 호출하여 데이터리스트가 비었는지 확인한다. 그 후 리스트에 아이디가 있는 지 검사한다. 아이디가 있으면 DuplicatedDataException에 에러 메시지를 담아서 던진다.

```java
    public void checkNullId(StudentList studentList, String studentId) throws NullDataException {
        this.checkNullData(studentList);
        for(int i = 0; i<studentList.getStudentCount(); i++) {
            if(studentList.getStudent(i).getStudentId().equals(studentId)) return;
        }
        throw new NullDataException("Student ID does not exist");
    }


    public void checkNullId(CourseList courseList, String id) throws NullDataException {
        this.checkNullData(courseList);
        for(int i = 0; i<courseList.getCourseCount(); i++) {
            if(courseList.getCourse(i).getCourseId().equals(id)) return;
        }
        throw new NullDataException("Course ID does not exist");
    }

    public void checkNullId(AccountList accountList, String accountInfo) throws NullDataException {
        this.checkNullData(accountList);
        StringTokenizer stringTokenizer = new StringTokenizer(accountInfo);
        String Id = stringTokenizer.nextToken();
        String password = stringTokenizer.nextToken();
        for(int i = 0; i<accountList.getAccountCount(); i++) {
            if(accountList.getAccount(i).getId().equals(Id)) {
                this.checkNullPassword(accountList.getAccount(i), password); return;
            }
        }
        throw new NullDataException("Account ID does not exist");
    }

    public void checkNullPassword(Account account, String password) throws NullDataException {
        if(account.getPassword().equals(password)) return;
        throw new NullDataException("Wrong Password");
    }
```

CheckNullId 함수는 데이터리스트와 아이디를 받고 checkNullData 함수를 호출하여 데이터리스트가 비었는지 확인한다. 그 후에 데이터리스트에 아이디가 없는지 확인한다. 없으면 NullDataException에 에러메시지를 담아서 던진다. 전달받는 데이터리스트가 AccountList일 경우에는 아이디와 비밀번호를 받는다. 그리고 아이디가 있으면 checkNullPassword 함수를 호출하여 비밀번호가 일치하는지 확인한다. 일치하지 않으면 NullDataException에 에러메시지를 담아서 던진다.

```java
    public void checkReservation(StudentList studentList, CourseList courseList,
            String studentId, String courseId) throws DuplicateDataException, NullDataException {
        this.checkNullData(studentList);
        this.checkNullData(courseList);
        this.checkNullId(studentList, studentId);
        this.checkNullId(courseList, courseId);
        Student student = null;
        Course course = null;
        for(int i = 0; i<studentList.getStudentCount(); i++) {
            if(studentList.getStudent(i).getStudentId().equals(studentId)) student = studentList.getStudent(i);
        }
        for(int i = 0; i<courseList.getCourseCount(); i++) {
            if(courseList.getCourse(i).getCourseId().equals(courseId)) course = courseList.getCourse(i);
        }
        this.checkDuplicateReservation(student, course);
        this.checkPrerequisite(student, course);
    }

    public void checkPrerequisite(Student student, Course course) throws NullDataException {
        List<String> completedCourseList = student.getCompletedCourseList();
        List<String> checkPrerequisiteList = course.getPrerequisiteList();
        if(!completedCourseList.containsAll(checkPrerequisiteList)) throw new NullDataException("Prerequisite Incompleted");

    }

    public void checkDuplicateReservation(Student student, Course course) throws DuplicateDataException {
        for(int i = 0; i<course.getReservedStudentCount(); i++) {
            if(course.getReservedStudent(i).equals(student.getStudentId())) throw new DuplicateDataException("Reservation is duplicated");
        }
    }
}
```

checkReservation 함수는 studentList, courseList, studentId, courseId를 받아서 checkNullData함수와 checkNullId 함수를 호출하여 리스트가 비었는지, 존재하지 않는 아이디인지 체크한다. 그리고 checkDuplicateReservation 함수를 호출한다. 그 후 checkPrerequisite 함수를 호출한다.

checkDuplicateReservation 함수는 Student와 Course를 받아 학생이 이미 수강 신청한 과목인지 체크한다. 이미 수강 신청한 과목이면 DuplicationDataException에 에러메시지를 담아 던진다.

checkPrerequisite 함수는 Student와 Course를 받아 학생의 이수완료과목이 강의의 선수과목을 포함하고 있는지 체크한다. 포함하고 있지 않다면 NullDataException에 에러메시지를 담아 전달한다.

AddService

```java
public class AddService extends AddServiceImplBase{

    private ExceptionChecker exceptionChecker;
    private GetServiceBlockingStub getServiceStub;
    private AddServiceBlockingStub addServiceStub;

    public AddService(ExceptionChecker exceptionChecker, GetServiceBlockingStub getServiceStub,
            AddServiceBlockingStub addServiceStub) {
        this.exceptionChecker = exceptionChecker;
        this.getServiceStub = getServiceStub;
        this.addServiceStub = addServiceStub;
    }

    @Override
    public void addStudent(DataInfo request, StreamObserver<ExceptionMessage> responseObserver) {
        StudentList studentList = this.getServiceStub.getAllStudent(UserId.newBuilder().build());
        ExceptionMessage.Builder exceptionMessage = ExceptionMessage.newBuilder();
        StringTokenizer stringTokenizer = new StringTokenizer(request.getInfo());
        String studentId = stringTokenizer.nextToken();
        try {
            this.exceptionChecker.checkDuplicateId(studentList, studentId);
            exceptionMessage = this.addServiceStub.addStudent(request).toBuilder();
            String methodName = Thread.currentThread().getStackTrace()[1].getMethodName();
            this.writeLog(request, methodName);
        } catch (DuplicateDataException e) {
            exceptionMessage.setMessage(e.getErrorMessage());
            e.printStackTrace();
        } catch (NullDataException e) {
            exceptionMessage.setMessage(e.getErrorMessage());
            e.printStackTrace();
        }
        responseObserver.onNext(exceptionMessage.build());
        responseObserver.onCompleted();
    }
```

```java
    private void writeLog(DataInfo request, String methodName) {
        DataInfo.Builder logInfo = DataInfo.newBuilder()
                .setUserId(request.getUserId())
                .setInfo(methodName + " " + LocalDateTime.now().toString());
        this.addServiceStub.addLog(logInfo.build());
    }
}
```

exceptionChecker와 Data로부터 함수를 요청하기 위한 getServiceStub, addServiceStub를 생성자 파라미터로 받아 전역변수에 담는다.

addStudent 함수는 우선 Data로부터 학생리스트를 받는다. 그리고 exceptionChecker의 checkDuplicateId 함수를 호출하여 학생리스트가 비었는지, Client로부터 받은 학생 아이디가 학생리스트에 이미 존재하는지 체크한다. 예외가 발생하면 catch문으로 넘어가서 ExceptionMessage에 에러메시지를 담는다. 그리고 ExceptionMessage를 Client에 반환한다. 예외가 발생하지 않으면 Data에게 학생 정보를 전달하며 AddStudent를 요청한다. 그리고 writeLog 함수를 호출한다. 마지막으로 Data로부터 받은 ExceptionMessage(AddStudent 요청에 대한 Data의 대답. 에러메지시가 비어있다.)를 Client에게 반환한다.

writeLog 함수는 Data에게 사용자의 아이디, 사용기능, 사용시간을 전달하면서 AddLog(로깅 정보

저장)를 요청한다.

```java
    @Override
    public void addCourse(DataInfo request, StreamObserver<ExceptionMessage> responseObserver) {
        CourseList courseList = this.getServiceStub.getAllCourse(UserId.newBuilder().build());
        ExceptionMessage.Builder exceptionMessage = ExceptionMessage.newBuilder();
        StringTokenizer stringTokenizer = new StringTokenizer(request.getInfo());
        String courseId = stringTokenizer.nextToken();
        try {
            this.exceptionChecker.checkDuplicateId(courseList, courseId);
            exceptionMessage = this.addServiceStub.addCourse(request).toBuilder();
            String methodName = Thread.currentThread().getStackTrace()[1].getMethodName();
            this.writeLog(request, methodName);
        } catch (DuplicateDataException e) {
            exceptionMessage.setMessage(e.getErrorMessage());
            e.printStackTrace();
        } catch (NullDataException e) {
            exceptionMessage.setMessage(e.getErrorMessage());
            e.printStackTrace();
        }
        responseObserver.onNext(exceptionMessage.build());
        responseObserver.onCompleted();
    }
```

addCourse함수는 addStudent함수와 똑 같은 구조다.

```java
    @Override
    public void makeReservation(DataInfo request, StreamObserver<ExceptionMessage> responseObserver) {
        ExceptionMessage.Builder exceptionMessage = ExceptionMessage.newBuilder();
        try {
            StudentList studentList = this.getServiceStub.getAllStudent(UserId.newBuilder().build());
            CourseList courseList = this.getServiceStub.getAllCourse(UserId.newBuilder().build());
            StringTokenizer stringTokenizer = new StringTokenizer(request.getInfo());
            String studentId = stringTokenizer.nextToken();
            String courseId = stringTokenizer.nextToken();
            this.exceptionChecker.checkReservation(studentList, courseList, studentId, courseId);
            exceptionMessage = this.addServiceStub.makeReservation(request).toBuilder();
            String methodName = Thread.currentThread().getStackTrace()[1].getMethodName();
            this.writeLog(request, methodName);
        } catch (NullDataException e) {
            exceptionMessage.setMessage(e.getErrorMessage());
            e.printStackTrace();
        } catch (DuplicateDataException e) {
            exceptionMessage.setMessage(e.getErrorMessage());
            e.printStackTrace();
        }
        responseObserver.onNext(exceptionMessage.build());
        responseObserver.onCompleted();
    }

    private void writeLog(DataInfo request, String methodName) {
        DataInfo.Builder logInfo = DataInfo.newBuilder()
                .setUserId(request.getUserId())
                .setInfo(methodName + " " + LocalDateTime.now().toString());
        this.addServiceStub.addLog(logInfo.build());
    }
}
```

makeReservation함수는 Data로부터 학생리스트, 강의리스트를 전달받는다. 그리고 Client로부터 받은 DataInfo에서 학생 아이디와 강의 아이디를 추출한다. 그 후 exceptionChecker의 checkReservation을 통해서 데이터 리스트가 비었는지, 존재하지 않는 아이디인지, 이미 수강신청한 과목인지, 선수과목을 만족하는지 체크한다. 예외가 발생하면 catch문으로 넘어가 ExceptionMessage에 에러메시지를 저장한다. 그리고 ExceptionMessage를 Client에게 반환한다. 예외가 발생하지 않으면 Data에게 학생 아이디와 강의 아이디를 전달하며 makeReservation을 요

청한다. 그리고 writeLog함수를 호출한다. 마지막으로 Data로부터 받은 ExceptionMessage(makeReservation 요청에 대한 Data의 대답. 에러메시지가 비어있다.)를 Client에게 반환한다.

DeleteService

```java
public class DeleteService extends DeleteServiceImplBase{

    private ExceptionChecker exceptionChecker;
    private GetServiceBlockingStub getServiceStub;
    private DeleteServiceBlockingStub deleteServiceStub;
    private AddServiceBlockingStub addServiceStub;

    public DeleteService(ExceptionChecker exceptionChecker,
            GetServiceBlockingStub getServiceStub, DeleteServiceBlockingStub deleteServiceStub, AddServiceBlockingStub addServiceStub) {
        this.exceptionChecker = exceptionChecker;
        this.getServiceStub = getServiceStub;
        this.deleteServiceStub = deleteServiceStub;
        this.addServiceStub = addServiceStub;
    }

    @Override
    public void deleteStudent(DataInfo request, StreamObserver<ExceptionMessage> responseObserver) {
        StudentList studentList = this.getServiceStub.getAllStudent(UserId.newBuilder().build());
        ExceptionMessage.Builder exceptionMessage = ExceptionMessage.newBuilder();
        try {
            this.exceptionChecker.checkNullId(studentList, request.getInfo());
            exceptionMessage = deleteServiceStub.deleteStudent(request).toBuilder();
            String methodName = Thread.currentThread().getStackTrace()[1].getMethodName();
            this.writeLog(request, methodName);
        } catch (NullDataException e) {
            exceptionMessage.setMessage(e.getErrorMessage());
            e.printStackTrace();
        }
        responseObserver.onNext(exceptionMessage.build());
        responseObserver.onCompleted();
    }
```

exceptionChecker와 Data로부터 함수를 요청하기 위한 getServiceStub, deleteServiceStub, addServiceStub를 생성자 파라미터로 받아 전역변수에 담는다.

deleteStudent 함수는 우선 Data로부터 학생리스트를 받는다. 그리고 exceptionChecker의 checkNullId함수를 호출하여 학생리스트가 비었는지, Client로부터 받은 학생 아이디가 학생 리스트에 존재하는지 체크한다. 예외가 발생하면 catch문으로 넘어가서 ExceptionMessage에 에러메시지를 담는다. 그리고 ExceptionMessage를 Client에 반환한다. 예외가 발생하지 않으면 Data에게 학생 아이디를 전달하며 deleteStudent를 요청한다. 그리고 writeLog 함수를 호출한다. 마지막으로 Data로부터 받은 ExceptionMessage(deleteStudent요청에 대한 Data의 대답. 에러 메지시가 비어있다.)를 Client에게 반환한다.

```java
    @Override
    public void deleteCourse(DataInfo request, StreamObserver<ExceptionMessage> responseObserver) {
        CourseList courseList = this.getServiceStub.getAllCourse(UserId.newBuilder().build());
        ExceptionMessage.Builder exceptionMessage = ExceptionMessage.newBuilder();
        try {
            this.exceptionChecker.checkNullId(courseList, request.getInfo());
            exceptionMessage = deleteServiceStub.deleteCourse(request).toBuilder();
            String methodName = Thread.currentThread().getStackTrace()[1].getMethodName();
            this.writeLog(request, methodName);
        } catch (NullDataException e) {
            exceptionMessage.setMessage(e.getErrorMessage());
            e.printStackTrace();
        }
        responseObserver.onNext(exceptionMessage.build());
        responseObserver.onCompleted();
    }

    private void writeLog(DataInfo request, String methodName) {
        DataInfo.Builder logInfo = DataInfo.newBuilder()
                .setUserId(request.getUserId())
                .setInfo(methodName + " " + LocalDateTime.now().toString());
        this.addServiceStub.addLog(logInfo.build());
    }
}
```

deleteCourse함수는 deleteStudent함수와 똑 같은 구조다.

GetService

```java
public class GetService extends GetServiceImplBase{

    private ExceptionChecker exceptionChecker;
    private GetServiceBlockingStub getServiceStub;
    private AddServiceBlockingStub addServiceStub;

    public GetService(ExceptionChecker exceptionChecker, GetServiceBlockingStub getServiceStub, AddServiceBlockingStub addServiceStub) {
        this.exceptionChecker = exceptionChecker;
        this.getServiceStub = getServiceStub;
        this.addServiceStub = addServiceStub;
    }

    @Override
    public void getAllStudent(UserId request, StreamObserver<StudentList> responseObserver) {
        StudentList.Builder studentList = this.getServiceStub.getAllStudent(request).toBuilder();
        try {
            this.exceptionChecker.checkNullData(studentList.build());
            String methodName = Thread.currentThread().getStackTrace()[1].getMethodName();
            this.writeLog(request, methodName);
        }catch (NullDataException e) {
            studentList.setException(e.getErrorMessage());
            e.printStackTrace();
        }
        responseObserver.onNext(studentList.build());
        responseObserver.onCompleted();
    }
```

exceptionChecker와 Data로부터 함수를 요청하기 위한 getServiceStub, addServiceStub를 생성자 파라미터로 받아 전역변수에 담는다.

getAllStudent 함수는 우선 Data로부터 학생리스트를 받는다. 그리고 exceptionChecker의 checkNullId함수를 호출하여 학생리스트가 비었는지 체크한다. 예외가 발생하면 catch문으로 넘어가서 학생리스트에 에러메시지를 담는다. 그리고 학생리스트를 Client에 반환한다. 예외가 발생하지 않으면 writeLog 함수를 호출한다. 그리고 학생리스트(에러메시지가 비어 있음.)를 Client로 반환한다.

```java
    public void getAllCourse(UserId request, StreamObserver<CourseList> responseObserver) {
        CourseList.Builder courseList = this.getServiceStub.getAllCourse(request).toBuilder();
        try {
            this.exceptionChecker.checkNullData(courseList.build());
            String methodName = Thread.currentThread().getStackTrace()[1].getMethodName();
            this.writeLog(request, methodName);
        }catch (NullDataException e) {
            courseList.setException(e.getErrorMessage());
            e.printStackTrace();
        }
        responseObserver.onNext(courseList.build());
        responseObserver.onCompleted();
    }

    public void writeLog(UserId request, String methodName) {
        DataInfo.Builder logInfo = DataInfo.newBuilder()
                .setUserId(request.getUserId())
                .setInfo(methodName + " " + LocalDateTime.now().toString());
        this.addServiceStub.addLog(logInfo.build());
    }
}
```

getAllCourse함수는 getAllStudent와 동일한 구조다.

LoginService

```java
public class LoginService extends LoginServiceImplBase{

    private ExceptionChecker exceptionChecker;
    private GetServiceBlockingStub getServiceStub;
    private AddServiceBlockingStub addServiceStub;

    public LoginService(ExceptionChecker exceptionChecker, GetServiceBlockingStub getServiceStub,
            AddServiceBlockingStub addServiceStub) {
        this.exceptionChecker = exceptionChecker;
        this.getServiceStub = getServiceStub;
        this.addServiceStub = addServiceStub;
    }

    @Override
    public void login(DataInfo request, StreamObserver<ExceptionMessage> responseObserver) {
        AccountList.Builder accountList = this.getServiceStub.getAllAccount(UserId.newBuilder().build()).toBuilder();
        ExceptionMessage.Builder exceptionMessage = ExceptionMessage.newBuilder();
        try {
            this.exceptionChecker.checkNullId(accountList.build(), request.getInfo());
            String methodName = Thread.currentThread().getStackTrace()[1].getMethodName();
            this.writeLog(request, methodName);
        }catch (NullDataException e) {
            exceptionMessage.setMessage(e.getErrorMessage());
            e.printStackTrace();
        }
        responseObserver.onNext(exceptionMessage.build());
        responseObserver.onCompleted();
    }
```

exceptionChecker와 Data로부터 함수를 요청하기 위한 getServiceStub, addServiceStub를 생성자 파라미터로 받아 전역변수에 담는다.

login함수는 우선 Data로부터 회원리스트를 받는다. 그리고 exceptionChecker의 checkNullId함수를 호출하여 회원리스트가 비었는지, Client로부터 받은 아이디가 회원리스트에 있는지 체크한다. 예외가 발생하면 catch문으로 넘어가서 ExceptionMessage에 에러메시지를 담는다. 그리고 ExceptionMessage를 Client에 반환한다. 예외가 발생하지 않으면 writeLog 함수를 호출한다. 그리

고 ExceptionMessage(에러메시지가 비어 있음)를 Client로 반환한다.

```java
    @Override
    public void signUp(DataInfo request, StreamObserver<ExceptionMessage> responseObserver) {
        AccountList.Builder accountList = this.getServiceStub.getAllAccount(UserId.newBuilder().build()).toBuilder();
        ExceptionMessage.Builder exceptionMessage = ExceptionMessage.newBuilder();
        StringTokenizer stringTokenizer = new StringTokenizer(request.getInfo());
        String accountId = stringTokenizer.nextToken();
        try {
            this.exceptionChecker.checkDuplicateId(accountList.build(), accountId);
            this.addServiceStub.addAccount(request);
            String methodName = Thread.currentThread().getStackTrace()[1].getMethodName();
            this.writeLog(request, methodName);
        }catch (NullDataException e) {
            exceptionMessage.setMessage(e.getErrorMessage());
            e.printStackTrace();
        } catch (DuplicateDataException e) {
            exceptionMessage.setMessage(e.getErrorMessage());
            e.printStackTrace();
        }
        responseObserver.onNext(exceptionMessage.build());
        responseObserver.onCompleted();
    }

    private void writeLog(DataInfo request, String methodName) {
        DataInfo.Builder logInfo = DataInfo.newBuilder()
                .setUserId(request.getUserId())
                .setInfo(methodName + " " + LocalDateTime.now().toString());
        this.addServiceStub.addLog(logInfo.build());
    }
}
```

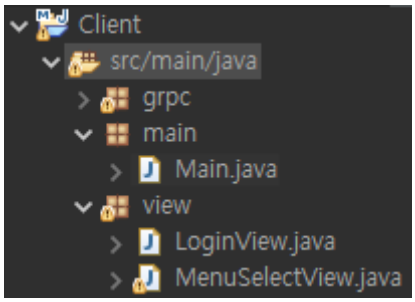
signUp 함수는 우선 Data로부터 회원리스트를 받는다. 그리고 exceptionChecker의 checkDuplicateId 함수를 호출하여 회원리스트가 비었는지, Client로부터 받은 회원 아이디가 회원리스트에 이미 존재하는지 체크한다. 예외가 발생하면 catch문으로 넘어가서 ExceptionMessage에 에러메시지를 담는다. 그리고 ExceptionMessage를 Client에 반환한다. 예외가 발생하지 않으면 Data에게 회원 정보를 전달하며 AddAccount를 요청한다. 그리고 writeLog 함수를 호출한다. 마지막으로 Data로부터 받은 ExceptionMessage(AddAccount 요청에 대한 Data의 대답. 에러메지시가 비어있다)를 Client에게 반환한다.

Main

```java
public class Main {

    public static void main(String[] args) {
        try {
            ManagedChannel channel = ManagedChannelBuilder.forAddress("localhost", 8080).usePlaintext().build();

            AddServiceBlockingStub addServiceStub = AddServiceGrpc.newBlockingStub(channel);
            DeleteServiceBlockingStub deleteServiceStub = DeleteServiceGrpc.newBlockingStub(channel);
            GetServiceBlockingStub getServiceStub = GetServiceGrpc.newBlockingStub(channel);

            ExceptionChecker exceptionChecker = new ExceptionChecker();

            Server server = ServerBuilder.forPort(8081)
                    .addService(new AddService(exceptionChecker, getServiceStub, addServiceStub))
                    .addService(new DeleteService(exceptionChecker, getServiceStub, deleteServiceStub, addServiceStub))
                    .addService(new GetService(exceptionChecker, getServiceStub, addServiceStub))
                    .addService(new LoginService(exceptionChecker, getServiceStub, addServiceStub))
                    .build();

            server.start();
            System.out.println("server is ready!!");
            System.out.println("port : 8081");
            server.awaitTermination();
        } catch (InterruptedException e) {e.printStackTrace();
        } catch (IOException e) {e.printStackTrace();}
    }
}
```

Data의 server에 접속하는 stub을 생성한다. 그리고 ExceptionCheck를 생성한다. Service에 stub과 ExceptionCheck를 생성자 파라미터로 넣어 생성하고 Client가 접속할 서버에 띄운다. 그리고 서버를 시작한다.

Client의 클래스



Main은 Server에 접속하는 stub을 만든다. 그리고 LoginView와 MenuSelelctView를 띄운다.

LoginView는 사용자에게 로그인 창을 보여주고 사용자가 입력하는 데이터를 받아 Server에게 요청하는 역할을 한다.

MenuSelectView는 사용자에게 메뉴 창을 보여주고 사용자가 입력하는 데이터를 받아 Server에게 서비스를 요청하는 역할을 한다.

LoginView

```java
public class LoginView {

    private LoginServiceBlockingStub loginServiceStub;
    private BufferedReader userInputReader;
    private String id;

    public LoginView(LoginServiceBlockingStub loginServiceStub, BufferedReader userInputReader) {
        this.loginServiceStub = loginServiceStub;
        this.userInputReader = userInputReader;
        this.id = "";
    }

    public String view() throws IOException {
        while(true) {
            printMenu();
            String userChoice = userInputReader.readLine().trim();
            switch(userChoice) {
                case "1": login();
                if(!this.id.isEmpty()) return this.id;
                break;
                case "2" : signUp(); break;
                default: System.err.println("Invalid Choice !!!");
            }
        }
    }
```

```java
    private void printMenu() {
        System.out.println("******************* Login *********************");
        System.out.println("1. Login");
        System.out.println("2. Sign Up");
    }
```

생성자 파라미터로 loginServiceStub과 BufferedReader타입의 변수 userInputReader를 받아 전역변수에 담는다. 전역변수 id는 빈 칸으로 설정한다.

view함수는 printMenu함수를 호출해 메뉴를 보여주고 사용자의 입력에 따라서 login함수와 signUp함수를 호출한다. 선택지에 없는 선택을 하면 다시 메뉴를 보여준다. login함수를 호출한 뒤 전역변수 id가 비어있으면 메뉴를 다시 보여주고, 비어있지 않으면 id를 리턴하고 함수를 끝낸다. 뒤에서 설명하겠지만, login함수에서 로그인에 성공하면 전역변수 id에 사용자가 입력한 id를 담는다.

```java
    private void signUp() throws IOException {
        String id = "";
        String password = "";
        String info = "";
        System.out.print("ID: "); id = userInputReader.readLine().trim();
        if(id.contains(" ")||id.isEmpty()) {System.err.println("blank is not allowed"); return;}
        else info += id + " ";
        System.out.print("Password: "); password = userInputReader.readLine().trim();
        if(password.contains(" ")||password.isEmpty()) {System.err.println("blank is not allowed"); return;}
        else info += password;
        DataInfo dataInfo = DataInfo.newBuilder().setUserId(id).setInfo(info).build();
        ExceptionMessage exceptionMessage = loginServiceStub.signUp(dataInfo);
        if(exceptionMessage.getMessage().isEmpty()) System.out.println("SUCEESS");
        else System.err.println(exceptionMessage.getMessage());
    }
    private void login() throws IOException {
        String id = "";
        String password = "";
        String info = "";
        System.out.print("ID: "); id = userInputReader.readLine().trim();
        if(id.contains(" ")||id.isEmpty()) {System.err.println("blank is not allowed"); return;}
        else info += id + " ";
        System.out.print("Password: "); password = userInputReader.readLine().trim();
        if(password.contains(" ")||password.isEmpty()) {System.err.println("blank is not allowed"); return;}
        else info += password;
        DataInfo dataInfo = DataInfo.newBuilder().setUserId(id).setInfo(info).build();
        ExceptionMessage exceptionMessage = loginServiceStub.login(dataInfo);
        if(exceptionMessage.getMessage().isEmpty()) { System.out.println("Hi " + id); this.id = id;}
        else System.err.println(exceptionMessage.getMessage());
    }
    private void printMenu() {
        System.out.println("******************** Login ********************");
        System.out.println("1. Login");
        System.out.println("2. Sign Up");
    }
```

signUp함수는 id와 password를 입력 받아 stub을 통해 Server에게 전달하면서 signUp을 요청한다. 그리고 Server로부터 요청의 결과로 받은 ExceptionMessage의 에러메시지를 체크한다. 에러메시지가 비어 있으면 성공한 것이므로 성공 메시지를 출력한다. 에러메시지가 비어 있지 않다면 ExceptionMessage의 에러메시지를 출력한다.

login함수는 id와 password를 입력 받아 stub을 통해 Server에게 전달하면서 login을 요청한다. 그리고 Server로부터 요청의 결과로 받은 ExceptionMessage의 에러메시지를 체크한다. 에러메시지가 비어 있으면 성공한 것이므로 전역변수 id에 사용자가 입력한 id를 담든다. 비어 있지 않다면 ExceptionMessage의 에러메시지를 출력한다.

MenuSelectView

```java
public class MenuSelectView {

    private GetServiceBlockingStub getServiceStub;
    private AddServiceBlockingStub addServicestub;
    private DeleteServiceBlockingStub deleteServicestub;
    private BufferedReader userInputReader;
    private String userId;

    public MenuSelectView(String userId, GetServiceBlockingStub getServiceStub, AddServiceBlockingStub addServicestub,
            DeleteServiceBlockingStub deleteServicestub, BufferedReader userInputReader) {
        this.getServiceStub = getServiceStub;
        this.addServicestub = addServicestub;
        this.deleteServicestub = deleteServicestub;
        this.userInputReader = userInputReader;
        this.userId = userId;
    }

    public void view() throws IOException {
        while(true) {
            printMenu();
            String userChoice = userInputReader.readLine().trim();
            switch(userChoice) {
                case "1": printAllStudent(); break;
                case "2" : printAllCourse(); break;
                case "3" : addStudent(); break;
                case "4" : deleteStudent(); break;
                case "5" : addCourse(); break;
                case "6" : deleteCourse(); break;
                case "7" : makeReservation(); break;
                case "x" : userInputReader.close(); return;
                default: System.err.println("Invalid Choice !!!");
            }
        }
    }
```

```java
    public void printMenu() {
        System.out.println("******************** MENU *********************");
        System.out.println("1. List Students");
        System.out.println("2. List Courses");
        System.out.println("3. Add Student");
        System.out.println("4. Delete Student");
        System.out.println("5. Add Course");
        System.out.println("6. Delete Course");
        System.out.println("7. Make Reservation");
        System.out.println("x. Exit");
    }
}
```

userId와 stub을 생성자 파라미터로 받아 전역 변수에 넣는다. view함수는 메뉴를 보여주는 printMenu함수를 호출하고 사용자에게 입력을 받아 메뉴를 고르도록 한다. 고른 메뉴에 따라서 다른 함수가 호출된다. 메뉴 외의 입력을 하면 다시 메뉴를 보여준다. X를 누르면 프로그램이 종료된다.

```java
public void printAllStudent() {
    StudentList studentList = getServiceStub.getAllStudent(UserId.newBuilder().build());
    if(!studentList.getException().isEmpty()) {
        System.err.println(studentList.getException());
        return;
    }
    System.out.println("******************** STUDENT LIST *********************");
    for(int i = 0; i<studentList.getStudentList().size(); i++) {
        Student student = studentList.getStudent(i);
        System.out.print(student.getStudentId() + " ");
        System.out.print(student.getName() + " ");
        System.out.print(student.getDepartment() + " ");
        for(int j = 0; j<student.getCompletedCourseList().size(); j++) {
            System.out.print(student.getCompletedCourse(j) + " ");
        }
        System.out.print("\n");
        System.out.print("Reserved Courses : ");
        for(int j = 0; j<student.getReservedCourseList().size(); j++) {
            System.out.print(student.getReservedCourse(j) + " ");
        }
        System.out.print("\n\n");
    }
}
```

printAllStudent함수는 stub을 통해 userId를 주면서 Server에게 getAllStudent를 요청한다. 받은 학생리스트에 에러메시지가 있는지 체크한다. 있으면 에러메시지를 출력하고 함수를 끝낸다. 없으면 학생리스트를 보기 좋게 가공하여 콘솔에 출력한다.

```java
    public void printAllCourse() {
        CourseList courseList = getServiceStub.getAllCourse(UserId.newBuilder().setUserId(this.userId).build());
        if(!courseList.getException().isEmpty()) {
            System.err.println(courseList.getException());
            return;
        }
        System.out.println("******************** COURSE LIST *********************");
        for(int i = 0; i<courseList.getCourseList().size(); i++) {
            Course course = courseList.getCourse(i);
            System.out.print(course.getCourseId() + " ");
            System.out.print(course.getProfessorName() + " ");
            System.out.print(course.getCourseName() + " ");
            for(int j = 0; j<course.getPrerequisiteList().size(); j++) {
                System.out.print(course.getPrerequisite(j) + " ");
            }
            System.out.print("\n");
            System.out.print("Reserved Students : ");
            for(int j = 0; j<course.getReservedStudentList().size(); j++) {
                System.out.print(course.getReservedStudent(j) + " ");
            }
            System.out.print("\n\n");
        }
    }
```

printAllCourse함수는 printAllStudent함수와 동일한 구조로 이루어져 있다.

```java
    public void addStudent() throws IOException {
        String studentInfo = "";
        String input = "";
        System.out.println("-----------Student Information----------");
        System.out.print("Student ID: "); input = userInputReader.readLine().trim();
        if(input.contains(" ")||input.isEmpty()) {System.err.println("blank is not allowed"); return;}
        else{studentInfo += input+" ";}
        System.out.print("Student Last Name: "); input = userInputReader.readLine().trim();
        if(input.contains(" ")||input.isEmpty()) {System.err.println("blank is not allowed"); return;}
        else{studentInfo += input+" ";}
        System.out.print("Student First Name: "); input = userInputReader.readLine().trim();
        if(input.contains(" ")||input.isEmpty()) {System.err.println("blank is not allowed"); return;}
        else{studentInfo += input+" ";}
        System.out.print("Student Department: "); input = userInputReader.readLine().trim();
        if(input.contains(" ")||input.isEmpty()) {System.err.println("blank is not allowed"); return;}
        else{studentInfo += input+" ";}
        System.out.println("**** Put Spacebar Between Courses ****");
        System.out.println("**** If you dont have any completed course, just press enter ****");
        System.out.print("Student Completed Courses: "); studentInfo += userInputReader.readLine().trim();
        DataInfo dataInfo = DataInfo.newBuilder().setUserId(this.userId).setInfo(studentInfo).build();
        ExceptionMessage exceptionMessage = addServicestub.addStudent(dataInfo);
        if(exceptionMessage.getMessage().isEmpty()) System.out.println("SUCEESS");
        else System.err.println(exceptionMessage.getMessage());
    }

    public void deleteStudent() throws IOException {
        System.out.print("Student ID: "); String input = userInputReader.readLine().trim();
        if(input.contains(" ")||input.isEmpty()) {System.err.println("blank is not allowed"); return;}
        DataInfo dataInfo = DataInfo.newBuilder().setUserId(this.userId).setInfo(input).build();
        ExceptionMessage exceptionMessage = deleteServicestub.deleteStudent(dataInfo);
        if(exceptionMessage.getMessage().isEmpty()) System.out.println("SUCEESS");
        else System.err.println(exceptionMessage.getMessage());
    }
```

addStudent함수는 사용자로부터 추가할 학생의 정보를 입력 받는다. 빈칸이거나 빈칸이 포함된 문자열을 입력할 경우 함수를 끝낸다. Stub을 통해 userId와 입력받은 정보를 주면서 Server에게 addStudent를 요청한다. Server로부터 받은 응답 ExceptionMessage에 에러메시지가 있는지 체크한다. 없으면 성공 메시지를 출력하고, 있으면 ExceptionMessage의 에러메시지를 출력한다.

deleteStudent함수는 사용자로부터 삭제할 학생의 아이디를 입력 받는다. 빈칸이거나 빈칸이 포함된 문자열을 입력할 경우 함수를 끝낸다. Stub을 통해 userId와 입력받은 정보를 주면서 Server에게 deleteStudent를 요청한다. Server로부터 받은 응답 ExceptionMessage에 에러메시지가 있는지 체크한다. 없으면 성공 메시지를 출력하고, 있으면 ExceptionMessage의 에러메시지를 출력한다.

```java
    public void addCourse() throws IOException {
        String courseInfo = "";
        String input = "";
        System.out.println("-----------Course Information----------");
        System.out.print("Course ID: "); input = userInputReader.readLine().trim();
        if(input.contains(" ")||input.isEmpty()) {System.err.println("blank is not allowed"); return;}
        else{courseInfo += input+" ";}
        System.out.print("Course Professor Name: "); input = userInputReader.readLine().trim();
        if(input.contains(" ")||input.isEmpty()) {System.err.println("blank is not allowed"); return;}
        else{courseInfo += input+" ";}
        System.out.print("Course Name: "); input = userInputReader.readLine().trim();
        if(input.contains(" ")||input.isEmpty()) {System.err.println("blank is not allowed"); return;}
        else{courseInfo += input+" ";}
        System.out.println("**** Put Spacebar Between Prerequisites ****");
        System.out.println("**** If you dont have any prerequisite, just press enter ****");
        System.out.print("Course Prerequisites: "); courseInfo += userInputReader.readLine().trim();
        DataInfo dataInfo = DataInfo.newBuilder().setUserId(this.userId).setInfo(courseInfo).build();
        ExceptionMessage exceptionMessage = addServicestub.addCourse(dataInfo);
        if(exceptionMessage.getMessage().isEmpty()) System.out.println("SUCEESS");
        else System.err.println(exceptionMessage.getMessage());
    }

    public void deleteCourse() throws IOException {
        System.out.print("Course ID: ");String input = userInputReader.readLine().trim();
        if(input.contains(" ")||input.isEmpty()) {System.err.println("blank is not allowed"); return;}
        DataInfo dataInfo = DataInfo.newBuilder().setUserId(this.userId).setInfo(input).build();
        ExceptionMessage exceptionMessage = deleteServicestub.deleteCourse(dataInfo);
        if(exceptionMessage.getMessage().isEmpty()) System.out.println("SUCEESS");
        else System.err.println(exceptionMessage.getMessage());
    }
```

addCourse함수, deleteCourse함수는 addStudent함수, deleteStudent함수와 동일한 구조로 설계되었다.

```java
    public void makeReservation() throws IOException {
        String reservationInfo = "";
        String input = "";
        System.out.print("Student ID: "); input = userInputReader.readLine().trim();
        if(input.contains(" ")||input.isEmpty()) {System.err.println("blank is not allowed"); return;}
        else{reservationInfo += input + " ";}
        System.out.print("Course ID: "); input = userInputReader.readLine().trim();
        if(input.contains(" ")||input.isEmpty()) {System.err.println("blank is not allowed"); return;}
        else{reservationInfo += input;}
        DataInfo dataInfo = DataInfo.newBuilder().setUserId(this.userId).setInfo(reservationInfo).build();
        ExceptionMessage exceptionMessage = addServicestub.makeReservation(dataInfo);
        if(exceptionMessage.getMessage().isEmpty()) System.out.println("SUCEESS");
        else System.err.println(exceptionMessage.getMessage());
    }
```

makeReservation함수는 사용자로부터 학생 아이디와 수강 신청할 강의 아이디를 입력 받는다. 빈 칸이거나 빈 칸이 포함된 문자열을 입력하면 함수를 끝낸다. Stub을 통해 사용자 아이디, 학생 아이디, 강의 아이디를 주면서 Server에게 makeReservation을 요청한다. Server로부터 받은 응답 ExceptionMessage에 에러메시지가 있는지 체크한다. 없으면 성공 메시지를 출력하고, 있으면 ExceptionMessage의 에러메시지를 출력한다.

```java
public class Main {

    public static void main(String[] args) throws IOException {
        ManagedChannel channel = ManagedChannelBuilder.forAddress("localhost", 8081).usePlaintext().build();
        GetServiceBlockingStub getServiceStub = GetServiceGrpc.newBlockingStub(channel);
        AddServiceBlockingStub addServicestub = AddServiceGrpc.newBlockingStub(channel);
        DeleteServiceBlockingStub deleteServiceStub = DeleteServiceGrpc.newBlockingStub(channel);
        LoginServiceBlockingStub loginServiceStub = LoginServiceGrpc.newBlockingStub(channel);
        BufferedReader userInputReader = new BufferedReader(new InputStreamReader(System.in));

        LoginView loginView = new LoginView(loginServiceStub, userInputReader);
        String userId = loginView.view();
        MenuSelectView menuSelectView = new MenuSelectView(userId, getServiceStub,addServicestub,deleteServiceStub,userInputReader);
        menuSelectView.view();
        channel.shutdown();
    }

}
```

Server의 서버에 접속할 stub을 만들고 생성자 파라미터로 stub을 넣어 LoginView를 생성한다. LoginView의 view함수를 호출하여 사용자 아이디를 리턴 값으로 받는다. 그리고 생성자 파라미터에 사용자 아이디와 stub을 넣어서 MenuSelectView를 생성한다. 마지막으로 menuSelectView의 view함수를 호출한다.

2. 실행 결과

```
******************** Login ********************
1. Login
2. Sign Up
```

Data, Server, Client 순으로 실행 시 나오는 Client의 콘솔창이다.

```
******************** Login ********************
1. Login
2. Sign Up
3
Invalid Choice !!!
```

메뉴 외의 입력을 하면 에러 메시지가 뜬다.

```
******************** Login ********************
1. Login
2. Sign Up
1
ID:
blank is not allowed
```

정보를 빈칸으로 입력하면 에러 메시지가 뜬다.

```
******************** Login ********************
1. Login
2. Sign Up
1
ID: 1 1
blank is not allowed
```

문자열 사이에 공백이 존재하면 에러가 뜬다.

```
******************** Login ********************
1. Login
2. Sign Up
1
ID: 1
Password: 1
Account ID does not exist
```

존재하지 않는 아이디를 입력하면 아이디가 존재하지 않는다는 에러 메시지가 출력된다.

```
******************** Login ********************
1. Login
2. Sign Up
1
ID: 20100123
Password: 2222
Wrong Password
```

존재하는 아이디를 입력하고 비밀번호를 틀리면 틀린 비밀번호라는 에러 메시지를 출력한다.

```
******************** Login ********************
1. Login
2. Sign Up
2
ID: a
Password: a
SUCEESS
```

회원가입을 하였다. 다시 a라는 아이디로 회원가입을 해보자.

```
1. Login
2. Sign Up
2
ID: a
Password: a
Account Id is duplicated
```

아이디가 중복되었다는 에러 메시지가 뜬다.

```
******************** Login ********************
1. Login
2. Sign Up
2
ID: 60181924
Password: 1111
SUCEESS
```

우선 회원가입을 한다.

```
******************** Login ********************
1. Login
2. Sign Up
1
ID: 60181924
Password: 1111
Hi 60181924
******************** MENU ********************
1. List Students
2. List Courses
3. Add Student
4. Delete Student
5. Add Course
6. Delete Course
7. Make Reservation
x. Exit
```

그리고 회원가입한 정보대로 로그인하면 메뉴창이 뜬다.

```
******************** MENU ********************
1. List Students
2. List Courses
3. Add Student
4. Delete Student
5. Add Course
6. Delete Course
7. Make Reservation
x. Exit
1
******************** STUDENT LIST ********************
20100123 Hwang Myunghan CS 12345 23456 17651 17652
Reserved Courses :

20090421 Kim Jason EE 17651 17652 17653 17654
Reserved Courses :

20110512 Park Hongsun CS 12345 23456 17652 17654
Reserved Courses :

20100123 Kim Yunmi CS 12345 17651 17652 17653
Reserved Courses :

20100125 Kim Hokyung ME 12345 23456 17651 17654
Reserved Courses :

20100323 Jung Philsoo CS 12345 23456 17651 17655
Reserved Courses :

20080678 Ahn Jonghyuk EE 12345 17651 17652 17654
Reserved Courses :

20110298 Lee Mijung ME 12345 23456 17651 17652
Reserved Courses :
```

1번을 입력하면 학생 리스트가 뜬다.

```
******************** MENU ********************
1. List Students
2. List Courses
3. Add Student
4. Delete Student
5. Add Course
6. Delete Course
7. Make Reservation
x. Exit
1
StudentList is null
******************** MENU ********************
1. List Students
2. List Courses
3. Add Student
4. Delete Student
5. Add Course
6. Delete Course
7. Make Reservation
x. Exit
```

만약 Students.txt에 내용이 없다면 다음과 같이 뜬다.

```
2
******************** COURSE LIST ********************
12345 Park Java_Programming
Reserved Students :

23456 Park C++_Programming
Reserved Students :

17651 Kim Models_of_Software_Systems 12345
Reserved Students :

17652 Ko Methods_of_Software_Development 23456
Reserved Students :

17653 Kim Managing_Software_Development
Reserved Students :

17654 Ahn Analysis_of_Software_Artifacts 17651
Reserved Students :

17655 Lee Architectures_of_Software_Systems 12345 17651
Reserved Students :
```

2번을 누르면 강의 리스트가 뜬다.

```
3
-----------Student Information----------
Student ID: 20100123
Student Last Name: 1
Student First Name: 1
Student Department: 1
**** Put Spacebar Between Courses ****
**** If you dont have any completed course, just press enter ****
Student Completed Courses: 1234 5678
Student ID is duplicated
```

3번을 눌러 학생을 추가하려고 할 때 이미 있는 학생 아이디를 입력하면 중복 에러 메시지를 출력한다.

```
3
-----------Student Information----------
Student ID: 60181924
Student Last Name: lee
Student First Name: woosung
Student Department: sw
**** Put Spacebar Between Courses ****
**** If you dont have any completed course, just press enter ****
Student Completed Courses: 1234 5678
SUCEESS
```

중복되지 않는 아이디를 입력하면 성공적으로 추가된다. 완료한 과목은 스페이스바로 구분하여 여러 개를 입력할 수 있다. 완료한 과목이 없을 경우에는 그냥 엔터를 누르면 된다.

```
-----------Student Information----------
Student ID: 41244
Student Last Name: kim
Student First Name: woosung
Student Department: sw
**** Put Spacebar Between Courses ****
**** If you dont have any completed course, just press enter ****
Student Completed Courses:
SUCEESS
******************** MENU ********************
1. List Students
```

완료한 과목이 없어도 추가가 되었다.

```
60181924 lee woosung sw 1234
Reserved Courses :

41244 kim woosung sw
Reserved Courses :
```

1번을 눌러서 학생 리스트를 확인해본 결과 잘 추가되었다.

```
******************** MENU ********************
1. List Students
2. List Courses
3. Add Student
4. Delete Student
5. Add Course
6. Delete Course
7. Make Reservation
x. Exit
4
Student ID: a
Student ID does not exist
```

학생을 삭제할 때 존재하지 않는 아이디를 입력하면 에러 메시지가 출력된다.

```
******************** MENU ********************
1. List Students
2. List Courses
3. Add Student
4. Delete Student
5. Add Course
6. Delete Course
7. Make Reservation
x. Exit
4
Student ID: 41244
SUCEESS
```

존재하는 아이디를 입력하면 삭제가 성공한다.

```
60181924 lee woosung sw 1234 5678
Reserved Courses :

******************** MENU ********************
```

1번을 눌러 아까 60181924 학생 아래에 있던 41244 학생이 삭제된 것을 확인할 수 있다.

```
-----------Course Information----------
Course ID: 1234
Course Professor Name: lee
Course Name: programming
**** Put Spacebar Between Prerequisites ****
**** If you dont have any prerequisite, just press enter ****
Course Prerequisites: 4567
SUCEESS
```

강의도 학생과 마찬가지로 중복되지 않은 아이디로 추가를 하면 성공적으로 추가된다.

```
17654 Ahn Analysis_of_Software_Artifacts 17651
Reserved Students :

17655 Lee Architectures_of_Software_Systems 12345 17651
Reserved Students :

1234 lee programming 4567
Reserved Students :
```

2번을 누르면 맨 아래에 추가된 강의를 볼 수 있다.

```
-----------Course Information----------
Course ID: 12345
Course Professor Name: 1
Course Name: 1
**** Put Spacebar Between Prerequisites ****
**** If you dont have any prerequisite, just press enter ****
Course Prerequisites: 1
Course ID is duplicated
```

중복된 강의를 추가하면 에러 메시지가 출력된다.

```
******************** MENU ********************
1. List Students
2. List Courses
3. Add Student
4. Delete Student
5. Add Course
6. Delete Course
7. Make Reservation
x. Exit
6
Course ID: 1234
SUCEESS
******************** MENU ********************
```

```
17655 Lee Architectures_of_Software_Systems 12345 17651
Reserved Students :

******************** MENU ********************
```

존재하는 강의를 삭제를 하고 2번을 눌러서 확인해보니 삭제가 된 것을 볼 수 있다.

```
******************** MENU ********************
1. List Students
2. List Courses
3. Add Student
4. Delete Student
5. Add Course
6. Delete Course
7. Make Reservation
x. Exit
6
Course ID: fsdf
Course ID does not exist
******************** MENU ********************
```

존재하지 않는 강의를 삭제하면 에러 메시지가 출력된다.

이제 수강신청을 해보자.

```
******************** MENU ********************
1. List Students
2. List Courses
3. Add Student
4. Delete Student
5. Add Course
6. Delete Course
7. Make Reservation
x. Exit
7
Student ID: 1111
Course ID: 12345
Student ID does not exist
******************** MENU ********************
```

존재하지 않는 학생 아이디를 입력하면 에러 메시지가 출력된다.

```
******************** MENU ********************
1. List Students
2. List Courses
3. Add Student
4. Delete Student
5. Add Course
6. Delete Course
7. Make Reservation
x. Exit
7
Student ID: 20100123
Course ID: vvvv
Course ID does not exist
******************** MENU ********************
```

존재하는 학생 아이디를 입력해도 존재하지 않는 강의 아이디를 입력하면 에러 메시지가 출력된다.

```
******************** MENU ********************
1. List Students
2. List Courses
3. Add Student
4. Delete Student
5. Add Course
6. Delete Course
7. Make Reservation
x. Exit
7
Student ID: 20100123
Course ID: 12345
SUCEESS
******************** MENU ********************
```

선수과목이 없는 강의를 수강신청했다.

```
20100123 Hwang Myunghan CS 12345 23456 17651 17652
Reserved Courses : 12345
```

학생 리스트를 보니 수강신청이 되었다.

```
12345 Park Java_Programming
Reserved Students : 20100123
```

강의 리스트에서도 확인할 수 있다.

```
1. List Students
2. List Courses
3. Add Student
4. Delete Student
5. Add Course
6. Delete Course
7. Make Reservation
x. Exit
7
Student ID: 20100123
Course ID: 12345
Reservation is duplicated
******************** MENU ********************
```

이미 수강신청한 과목을 또 수강 신청하면 오류 메시지가 출력된다.

이제 선수과목이 있는 강의를 수강신청해보자.

```
17655 Lee Architectures_of_Software_Systems 12345 17651
Reserved Students :
```

17655 강의는 선수과목이 12345 17651이다.

```
20070452 Ko Kyungmin CS 12345 17652 17655
Reserved Courses :
```

20070452 학생은 이수과목이 12345 17652 17655다.

```
******************** MENU ********************
1. List Students
2. List Courses
3. Add Student
4. Delete Student
5. Add Course
6. Delete Course
7. Make Reservation
x. Exit
7
Student ID: 20070452
Course ID: 17655
Prerequisite Incompleted
******************** MENU ********************
```

선수과목을 이수하지 않았기 때문에 에러 메시지가 뜬다.

```
20120808 Kim Sungsuk EE 12345 23456 17651 17652
Reserved Courses :
```

```
******************** MENU ********************
1. List Students
2. List Courses
3. Add Student
4. Delete Student
5. Add Course
6. Delete Course
7. Make Reservation
x. Exit
7
Student ID: 20120808
Course ID: 17655
SUCEESS
******************** MENU ********************
```

선이수 과목을 모두 완료한 학생은 수강 신청에 성공하였다.

```
20120808 Kim Sungsuk EE 12345 23456 17651 17652
Reserved Courses : 17655
```

```
17655 Lee Architectures_of_Software_Systems 12345 17651
Reserved Students : 20120808
```

학생 리스트와 강의 리스트에서도 확인할 수 있다.

```
        at grpc.AddServiceGrpc$MethodHandlers.invoke(AddServiceGrpc.java:497)
        at io.grpc.stub.ServerCalls$UnaryServerCallHandler$UnaryServerCallListener.onHalfClose(ServerCalls.java:171)
        at io.grpc.internal.ServerCallImpl$ServerStreamListenerImpl.halfClosed(ServerCallImpl.java:283)
        at io.grpc.internal.ServerImpl$JumpToApplicationThreadServerStreamListener$1HalfClosed.runInContext(ServerImpl.java:707)
        at io.grpc.internal.ContextRunnable.run(ContextRunnable.java:37)
        at io.grpc.internal.SerializingExecutor.run(SerializingExecutor.java:123)
        at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1149)
        at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
        at java.lang.Thread.run(Thread.java:750)
exception.NullDataException: Prerequisite Incompleted
        at exceptionChecker.ExceptionChecker.checkPrerequisite(ExceptionChecker.java:105)
        at exceptionChecker.ExceptionChecker.checkReservation(ExceptionChecker.java:99)
        at service.AddService.makeReservation(AddService.java:85)
        at grpc.AddServiceGrpc$MethodHandlers.invoke(AddServiceGrpc.java:497)
        at io.grpc.stub.ServerCalls$UnaryServerCallHandler$UnaryServerCallListener.onHalfClose(ServerCalls.java:171)
        at io.grpc.internal.ServerCallImpl$ServerStreamListenerImpl.halfClosed(ServerCallImpl.java:283)
        at io.grpc.internal.ServerImpl$JumpToApplicationThreadServerStreamListener$1HalfClosed.runInContext(ServerImpl.java:707)
        at io.grpc.internal.ContextRunnable.run(ContextRunnable.java:37)
        at io.grpc.internal.SerializingExecutor.run(SerializingExecutor.java:123)
        at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1149)
        at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
        at java.lang.Thread.run(Thread.java:750)
```

에러가 발생할 때마다 Server 콘솔창에서 상세 에러 내용을 출력하고 있다. 이를 통해 에러가 발생한 위치를 확인할 수 있다. Server는 종료되지 않고 정상적으로 실행되고 있다.

```
60181924 getAllCourse 2022-11-07T22:44:27.992
60181924 deleteCourse 2022-11-07T22:44:32.780
60181924 getAllCourse 2022-11-07T22:44:34.233
60181924 getAllStudent 2022-11-07T22:45:01.277
60181924 login 2022-11-07T22:49:58.230
60181924 getAllStudent 2022-11-07T22:50:11.317
60181924 signUp 2022-11-07T22:52:24.165
60181924 login 2022-11-07T22:53:08.132
60181924 addStudent 2022-11-07T22:53:29.603
60181924 getAllStudent 2022-11-07T22:53:30.704
60181924 addStudent 2022-11-07T22:53:52.707
60181924 getAllStudent 2022-11-07T22:54:30.749
60181924 deleteStudent 2022-11-07T22:55:09.094
60181924 getAllStudent 2022-11-07T22:55:33.028
60181924 addCourse 2022-11-07T22:56:41.538
60181924 getAllCourse 2022-11-07T22:56:44.088
60181924 deleteCourse 2022-11-07T22:57:07.610
60181924 getAllCourse 2022-11-07T22:57:08.438
60181924 makeReservation 2022-11-07T22:58:16.865
60181924 getAllStudent 2022-11-07T22:58:21.261
60181924 getAllCourse 2022-11-07T22:58:26.262
60181924 makeReservation 2022-11-07T22:59:27.146
60181924 getAllStudent 2022-11-07T22:59:33.214
60181924 getAllCourse 2022-11-07T22:59:40.175
20090421 login 2022-11-07T23:01:55.895
20090421 getAllStudent 2022-11-07T23:01:57.331
20090421 getAllCourse 2022-11-07T23:01:57.968
20090421 addStudent 2022-11-07T23:02:05.244
```

Data의 log.txt파일이다. 사용자가 서비스를 성공적으로 사용할 때마다 log.txt에 사용자 아이디, 사용한 서비스, 사용한 시간이 기록한다. 60181924 회원으로 사용한 서비스 기록이 작성된 것을 볼 수 있다.

```
******************** MENU ********************
1. List Students
2. List Courses
3. Add Student
4. Delete Student
5. Add Course
6. Delete Course
7. Make Reservation
x. Exit
x
```

X를 누르면 프로그램이 종료된다.

3. 결론

GRPC를 사용하여 클라이언트 서버 프로그램을 구현해보았다. 이 과정을 통해서 클라이언트와 서버의 역할과 서로 상호작용하는 방식에 대해서 학습할 수 있었다. 그리고 분산 시스템에 대해서 이해하고 어떻게 구현해야 하는지 고민할 수 있었다. 이번 기회로 GRPC라는 새로운 프레임워크를 학습할 수 있었고 이번 경험이 또 다른 프레임워크를 사용할 때 좀 더 수월하도록 도와줄 것이다. 그리고 가시성 있는 코드를 작성하고 단번에 이해되는 변수명을 정하는 방법에 대해서 많이 고민하고 배울 수 있었다. 부족하다고 느꼈던 것은 간단하고 명료한 구조로 프로그램을 설계하는 능력이다. 개발을 끝내고 내 프로그램의 구조를 보았을 때 복잡하고 어수선하다는 것을 발견하였다. 프로그램 구조를 설계하는 부분은 앞으로 분발이 필요하다. 시스템을 개발하면서 들었던 또 다른 고민은 다수의 사용자가 프로그램을 사용할 때 데이터를 관리하는 방법이었다. 학사관리 시스템처럼 다수의 사용자가 동시에 사용하는 경우 분산 시스템에서는 어디를 크리티컬 섹션으로 설정하고 어떻게 크리티컬 섹션을 관리해야 하는지에 대한 고민이 들었다. 이러한 고민을 해결하기 위해서는 앞으로 더 많은 학습과 경험을 통해 분산 시스템에 대한 깊은 이해도를 갖춰야 하겠다.